

LESBOEK

# API's bouwen

## met vanilla PHP

---

*Van je eerste HTTP-request tot je eigen REST API*

HTTP · JSON · REST · Statuscodes · Headers · CORS · Routes · Composer

**Eindproject:** Een forum-API

Een REST API voor threads, topics en replies, gebouwd zonder framework

# Inhoudsopgave

---

|                                                              |    |
|--------------------------------------------------------------|----|
| Inhoudsopgave .....                                          | 1  |
| Voorwoord .....                                              | 5  |
| Hoe werk je met dit boek? .....                              | 5  |
| Code in dit boek .....                                       | 6  |
| Wat heb je nodig? .....                                      | 6  |
| 1. Wat is een API? .....                                     | 7  |
| Een ober in een restaurant .....                             | 7  |
| Voorbeelden van API's .....                                  | 7  |
| Website of API: wat is het verschil? .....                   | 7  |
| Frontend en backend .....                                    | 8  |
| Wat gaan we bouwen? .....                                    | 9  |
| 2. Het HTTP-protocol: hoe praat het web? .....               | 10 |
| Wat is een protocol? .....                                   | 10 |
| Request en response .....                                    | 10 |
| Een URL uit elkaar gehaald .....                             | 11 |
| Hoe ziet een request eruit? .....                            | 11 |
| Hoe ziet een response eruit? .....                           | 12 |
| Zelf kijken: de Network-tab .....                            | 13 |
| 3. Je werkplek inrichten .....                               | 14 |
| XAMPP .....                                                  | 14 |
| De map htdocs .....                                          | 15 |
| Visual Studio Code .....                                     | 15 |
| Je eerste mini-API .....                                     | 15 |
| Postman .....                                                | 16 |
| De database klaarzetten .....                                | 16 |
| 4. HTTP-methodes: wat wil je doen? .....                     | 18 |
| Werkwoorden van het web .....                                | 18 |
| CRUD .....                                                   | 18 |
| De methodes één voor één .....                               | 18 |
| Veilig en idempotent .....                                   | 20 |
| De methode uitlezen in PHP .....                             | 20 |
| 5. Headers: extra informatie bij requests en responses ..... | 23 |
| Wat zijn headers? .....                                      | 23 |
| Belangrijke request-headers .....                            | 23 |
| Belangrijke response-headers .....                           | 24 |
| Content-Type: de belangrijkste header .....                  | 24 |

|                                                  |    |
|--------------------------------------------------|----|
| Headers versturen in PHP .....                   | 25 |
| Request-headers uitlezen in PHP .....            | 26 |
| 6. JSON: de taal van API's .....                 | 28 |
| Wat is JSON? .....                               | 28 |
| De regels van JSON .....                         | 28 |
| JSON in PHP .....                                | 29 |
| JSON ontvangen in een API .....                  | 30 |
| 7. HTTP-responses en statuscodes .....           | 32 |
| Hoe ging het? .....                              | 32 |
| De vijf groepen .....                            | 32 |
| De belangrijkste statuscodes .....               | 32 |
| Best practices voor foutcodes .....              | 34 |
| Statuscodes in PHP .....                         | 35 |
| 8. RESTful API's .....                           | 37 |
| Wat is REST? .....                               | 37 |
| De belangrijkste principes .....                 | 37 |
| URL's ontwerpen: de regels .....                 | 37 |
| Een collectie en een item .....                  | 38 |
| Versies .....                                    | 39 |
| 9. Best practices voor response-structuren ..... | 40 |
| Waarom een vaste structuur? .....                | 40 |
| De envelop: meta en data .....                   | 40 |
| Fouten: een vaste foutstructuur .....            | 41 |
| De regels op een rij .....                       | 41 |
| 10. Routes en .htaccess .....                    | 43 |
| Mooie URL's .....                                | 43 |
| Eén voordeur: de front controller .....          | 43 |
| Wat is .htaccess? .....                          | 43 |
| Reguliere expressies in het kort .....           | 44 |
| De .htaccess van onze API .....                  | 44 |
| Routes: welke code bij welke URL? .....          | 46 |
| 11. Composer, namespaces en PSR-4 .....          | 48 |
| Het probleem: veel bestanden .....               | 48 |
| Namespaces .....                                 | 48 |
| PSR-4: een afspraak over mappen .....            | 49 |
| Composer .....                                   | 50 |
| 12. CORS: wie mag je API gebruiken? .....        | 54 |
| De same-origin policy .....                      | 54 |
| CORS: toestemming geven .....                    | 54 |

|                                                      |    |
|------------------------------------------------------|----|
| Simpele requests en de preflight .....               | 55 |
| CORS in onze forum-API.....                          | 56 |
| Zelf proberen .....                                  | 57 |
| 13. Eindproject: de forum-API .....                  | 59 |
| Wat gaan we bouwen? .....                            | 59 |
| De mappenstructuur.....                              | 60 |
| Wat is er veranderd aan het originele project? ..... | 61 |
| Stap 1: het project aanmaken.....                    | 62 |
| Stap 2: index.php, de voordeur .....                 | 63 |
| Stap 3: de database-klasse.....                      | 64 |
| Stap 4: de models .....                              | 66 |
| Stap 5: ApiResponse, de responses .....              | 69 |
| Stap 6: de controllers.....                          | 72 |
| Stap 7: de routetabel .....                          | 79 |
| Stap 8: de RequestHandler, de router .....           | 79 |
| Hoe werkt alles samen? .....                         | 83 |
| 14. De API testen met Postman .....                  | 85 |
| Waarom testen? .....                                 | 85 |
| Een collection en een variabele .....                | 85 |
| Requests maken.....                                  | 85 |
| Een testlijst .....                                  | 86 |
| Automatische tests in Postman .....                  | 87 |
| 15. Afronding en verder leren.....                   | 89 |
| Wat heb je geleerd?.....                             | 89 |
| Hoe ga je verder?.....                               | 89 |
| Bijlage A: Antwoorden .....                          | 91 |
| Hoofdstuk 1.....                                     | 91 |
| Hoofdstuk 2.....                                     | 91 |
| Hoofdstuk 3.....                                     | 92 |
| Hoofdstuk 4.....                                     | 92 |
| Hoofdstuk 5.....                                     | 93 |
| Hoofdstuk 6.....                                     | 94 |
| Hoofdstuk 7.....                                     | 94 |
| Hoofdstuk 8.....                                     | 95 |
| Hoofdstuk 9.....                                     | 96 |
| Hoofdstuk 10.....                                    | 97 |
| Hoofdstuk 11.....                                    | 97 |
| Hoofdstuk 12.....                                    | 98 |
| Hoofdstuk 13.....                                    | 99 |

|                                 |     |
|---------------------------------|-----|
| Hoofdstuk 14.....               | 103 |
| Bijlage B: Begrippenlijst ..... | 104 |

# Voorwoord

---

Welkom bij dit lesboek over het bouwen van een **API** met **PHP**! Misschien heb je al eens een website gemaakt met PHP, waarbij PHP een HTML-pagina maakt die in de browser verschijnt. In dit boek doe je iets anders. Je bouwt een programma dat geen pagina's maakt, maar **gegevens** levert aan andere programma's. Zo'n programma heet een API.

API's zijn overal. De weer-app op je telefoon, de kaart in een navigatie-app en de berichten in een chat-app: ze halen hun gegevens allemaal op bij een API. Als je weet hoe je zelf een API bouwt, begrijp je hoe een groot deel van het internet werkt.

We bouwen de API met **vanilla PHP**. *Vanilla* (vanille, de meest gewone smaak ijs) betekent: gewoon PHP, zonder framework zoals Laravel. Dat is meer werk, maar daardoor zie je precies hoe alles werkt. Als je later met een framework werkt, begrijp je wat het achter de schermen voor je doet.

Aan het einde van dit boek heb je een complete API gebouwd voor een **forum**. Met die API kan een app threads, topics en replies ophalen, aanmaken, wijzigen en verwijderen.

## Hoe werk je met dit boek?

Het boek is zo opgebouwd dat elk hoofdstuk voortbouwt op het vorige. De eerste hoofdstukken leggen uit hoe het internet en API's werken. Daarna leer je stap voor stap de onderdelen die je nodig hebt, en in hoofdstuk 13 bouw je de hele forum-API.

In het boek kom je verschillende kaders tegen:



### Tip

Handige trucs die je werk makkelijker maken.



### Let op

Dingen waar beginners vaak een fout maken. Lees deze kaders goed!



### Goed om te weten

Extra achtergrondinformatie. Leuk om te weten, maar niet verplicht.



### Opdracht

Aan het einde van elk hoofdstuk staan opdrachten. Maak ze echt, want programmeren leer je alleen door het zelf te doen. De antwoorden staan achterin het boek, in **Bijlage A**. Kijk pas naar het antwoord als je het zelf hebt geprobeerd!

## Code in dit boek

Code staat in grijze blokken met kleurtjes. Die kleurtjes heten **syntax highlighting**. Ze helpen je om snel te zien wat wat is. Boven veel codeblokken staat de naam van het bestand waar de code in hoort, zoals hieronder.

 voorbeeld.php

```
<?php
// Dit is commentaar: PHP slaat dit over
$naam = 'Sam';
echo 'Hallo ' . $naam;
```

Als je code overtypet, let dan goed op hoofdletters, puntkomma's, haakjes en aanhalingstekens. Een computer is heel precies.

## Wat heb je nodig?

- Een laptop of computer met Windows of macOS.
- Een internetverbinding (om programma's te downloaden).
- Een beetje basiskennis van PHP: variabelen, **if**, lussen, functies en arrays. Klassen en objecten komen in dit boek ook voor; die leggen we uit waar dat nodig is.
- Een beetje kennis van databases en SQL (**SELECT**, **INSERT**, **UPDATE**, **DELETE**).
- Zin om te puzzelen!

Veel plezier en succes!

# 1. Wat is een API?

## Een ober in een restaurant

Stel je voor dat je in een restaurant zit. Jij wilt eten, en in de keuken staat de kok die het eten maakt. Maar je loopt niet zelf de keuken in om te vertellen wat je wilt. Daar is de **ober** voor. De ober neemt je bestelling op, brengt die naar de keuken en komt terug met je eten.

Een **API** werkt precies zo. API staat voor **Application Programming Interface**. In gewone taal: een vaste manier waarop twee programma's met elkaar kunnen praten.

- Jij ben de **client**: het programma dat iets wil hebben, bijvoorbeeld een app op je telefoon.
- De **keuken** is de **server** met de database: daar liggen de gegevens.
- De **ober** is de **API**: die neemt het verzoek aan, haalt de gegevens op en brengt ze terug.

En net als in een restaurant zijn er afspraken. Je kunt alleen bestellen wat er op de **menukaart** staat, en je moet het op de goede manier vragen. Bij een API staat op de menukaart welke gegevens je kunt opvragen en hoe je dat doet. Die menukaart heet de **documentatie** van de API.

## Voorbeelden van API's

Je gebruikt elke dag API's zonder het te merken:

- De **weer-app** op je telefoon heeft zelf geen weerstation. Hij vraagt het weerbericht op bij de API van een weerdienst.
- Als je op een website kunt **inloggen met Google**, praat die website met de API van Google.
- Een **webwinkel** laat je betalen via iDEAL. De webwinkel stuurt je betaling door naar de API van een betaaldienst.
- Een **game** bewaart je score en je vrienden via een API op een server, zodat je op een andere telefoon verder kunt spelen.

## Website of API: wat is het verschil?

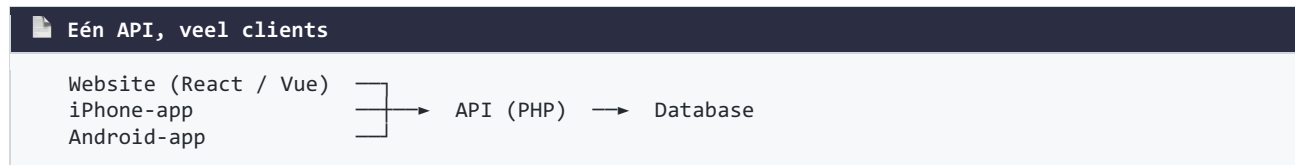
Als je een gewone website bezoekt, stuurt de server een **HTML-pagina** terug. Die is bedoeld voor mensen: met opmaak, plaatjes en knoppen.

Een API stuurt **alleen gegevens** terug, zonder opmaak. Die gegevens zijn bedoeld voor een ander programma. Dat programma bepaalt zelf hoe het de gegevens op het scherm zet. Meestal gebruikt een API daarvoor het formaat **JSON**. Zo ziet een antwoord van een weer-API er bijvoorbeeld uit:

 Antwoord van een weer-API

```
{
  "stad": "Groningen",
  "temperatuur": 17,
  "weer": "bewolkt"
}
```

Het mooie hiervan is dat **veel verschillende programma's** dezelfde API kunnen gebruiken. De website, de iPhone-app en de Android-app van een bedrijf halen hun gegevens vaak allemaal bij dezelfde API op. De API hoeft maar één keer gebouwd te worden.



## Frontend en backend

Je hoort programmeurs vaak praten over **frontend** en **backend**.

- De **frontend** is alles wat de gebruiker ziet: de website of de app. Denk aan HTML, CSS, JavaScript, React of Vue.
- De **backend** is alles wat achter de schermen gebeurt: de server, de database en de API. Denk aan PHP en MySQL.

De API is de brug tussen frontend en backend. In dit boek bouw je de backend: een API in PHP.

## Wat gaan we bouwen?

In dit boek bouw je een API voor een **forum**. Een forum is een website waar mensen met elkaar discussiëren. Ons forum bestaat uit drie soorten gegevens:

- **Threads:** de grote onderwerpen, zoals *Games* of *Huiswerkhulp*.
- **Topics:** de gesprekken binnen een thread, zoals *Welke game moet ik kopen?*. Elke topic hoort bij één thread.
- **Replies:** de reacties op een topic. Elke reply hoort bij één topic.

### Opbouw van het forum

```
Thread (bijv. "Games")
└─ Topic (bijv. "Welke game moet ik kopen?")
    └─ Reply (bijv. "Ik zou Minecraft nemen!")
```

Daarnaast zijn er **users**: de mensen die threads, topics en replies schrijven. Met onze API kan een app al deze gegevens ophalen, nieuwe toevoegen, wijzigen en verwijderen.

### Samenvatting

- Een **API** (Application Programming Interface) is een vaste manier waarop programma's met elkaar praten.
- De **client** vraagt iets, de **server** geeft antwoord. De API is de ober daartussen.
- Een website stuurt HTML voor mensen; een API stuurt gegevens (meestal **JSON**) voor programma's.
- De **frontend** is wat de gebruiker ziet, de **backend** werkt achter de schermen.
- In dit boek bouw je de backend: een forum-API in PHP.

### Opdracht: Hoofdstuk 1

1. Leg in je eigen woorden uit wat een API is. Gebruik een eigen vergelijking, dus niet het restaurant.
2. Noem drie apps die jij gebruikt en die volgens jou gegevens ophalen bij een API. Welke gegevens halen ze op?
3. Wat is het verschil tussen wat een gewone website terugstuurt en wat een API terugstuurt?
4. Waarom is het handig dat de website en de apps van een bedrijf dezelfde API gebruiken?
5. Bedenk voor een webwinkel welke soorten gegevens een API zou moeten kunnen leveren. Noem er minstens vier.

## 2. Het HTTP-protocol: hoe praat het web?

### Wat is een protocol?

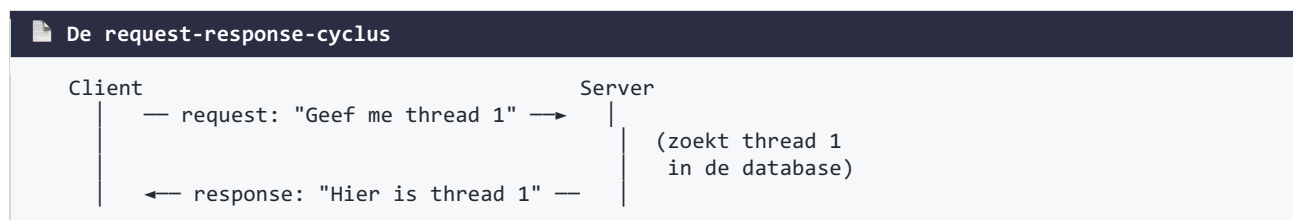
Als je iemand belt, zijn er ongeschreven regels. Degene die opneemt zegt eerst iets, dan zeg jij wie je bent, en aan het einde zeggen jullie allebei *doei*. Als iemand zich niet aan die regels houdt, wordt het gesprek raar.

Computers hebben ook zulke afspraken nodig. Een vaste set afspraken over hoe computers met elkaar communiceren heet een **protocol**. Het protocol van het web heet **HTTP**: *HyperText Transfer Protocol*. Elke keer als je een website bezoekt of een app gegevens ophaalt, wordt HTTP gebruikt.

### Request en response

HTTP werkt altijd op dezelfde manier: er is een vraag en een antwoord.

1. De **client** stuurt een vraag naar de server. Die vraag heet een **request** (verzoek).
2. De **server** verwerkt de vraag en stuurt een antwoord terug. Dat antwoord heet een **response**.



Belangrijk: de server begint nooit zelf een gesprek. Hij wacht altijd op een request. En elk request staat helemaal op zichzelf. De server onthoudt niet wat je bij het vorige request vroeg. Dat noemen we **stateless** (zonder toestand). Moet de server iets weten, zoals wie je bent, dan moet je dat bij elk request opnieuw meesturen.



#### Een POST-request met body

```
POST /forum-api/threads HTTP/1.1
Host: localhost
Content-Type: application/json
Content-Length: 74

{"title": "Huiswerkhulp", "content": "Stel hier je vragen!", "user_id": 1}
```

## Hoe ziet een response eruit?

Een response is ook tekst, met ook drie delen:

1. De **statusregel**: de versie van HTTP, een **statuscode** (een getal) en een korte tekst.
2. De **headers**.
3. Een lege regel, en daarna de **body**: het antwoord zelf.

#### Een response

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
  "data": {
    "id": 1,
    "title": "Eum omnis unde quaerat et.",
    "user_id": 11
  }
}
```

De **statuscode** vertelt in één getal hoe het ging. **200** betekent *alles is gelukt*, **404** betekent *niet gevonden*. In hoofdstuk 7 leer je alle belangrijke statuscodes.

## Zelf kijken: de Network-tab

Je kunt zelf zien welke requests je browser verstuurt. Open een website, druk op **F12** en klik op het tabblad **Network** (Netwerk). Ververs de pagina. Je ziet nu een lijst met alle requests die je browser doet: voor de HTML, de CSS, de plaatjes en vaak ook voor API's. Klik op een request om de headers, de statuscode en de response te bekijken.



### Tip

Filter in de Network-tab op **Fetch/XHR**. Dan zie je alleen de requests die JavaScript doet. Dat zijn vaak de requests naar API's. Probeer het eens op een nieuwssite of op YouTube.



### Samenvatting

- Een **protocol** is een set afspraken over hoe computers communiceren. Het web gebruikt **HTTP**.
- De client stuurt een **request**, de server stuurt een **response** terug.
- HTTP is **stateless**: elk request staat op zichzelf.
- Een **URL** bestaat uit scheme, host, path en eventueel een query string.
- Een request heeft een request-regel (met de **methode**), **headers** en soms een **body**.
- Een response heeft een statusregel (met de **statuscode**), **headers** en meestal een **body**.



### Opdracht: Hoofdstuk 2

1. Wat betekent de afkorting HTTP, en wat is een protocol?
2. Leg uit wat bedoeld wordt met *HTTP is stateless*.
3. Haal deze URL uit elkaar in scheme, host, path en query string:  
`https://api.weer.nl/v1/steden/groningen?dagen=3&eenheid=celsius`
4. Uit welke drie delen bestaat een HTTP-request? En een response?
5. Open de Network-tab van je browser op een website naar keuze. Zoek een request met statuscode 200 en een request met een andere statuscode (bijvoorbeeld 304 of 404). Schrijf van beide de URL en de statuscode op.

### 3. Je werkplek inrichten

Voordat je een API kunt bouwen, heb je een paar programma's nodig. Dit is eenmalig werk. In dit hoofdstuk installeer je XAMPP, zet je de database klaar, installeer je VS Code en Postman, en maak je je allereerste mini-API.

#### XAMPP

Een PHP-bestand kun je niet zomaar in je browser openen, zoals een HTML-bestand. Er moet een **webserver** zijn die het PHP-bestand uitvoert en het resultaat terugstuurt. Voor de database heb je een **databaseserver** nodig.

**XAMPP** is een gratis pakket dat alles in één keer installeert:

- **Apache:** de webserver. Apache ontvangt de HTTP-requests en stuurt de responses terug.
- **MariaDB:** de databaseserver. MariaDB werkt precies hetzelfde als MySQL. In XAMPP wordt het ook gewoon *MySQL* genoemd.
- **PHP:** de programmeertaal die je code uitvoert.
- **phpMyAdmin:** een website waarmee je je database kunt beheren.

Zo installeer je XAMPP:

1. Ga naar **apachefriends.org** en download XAMPP voor jouw besturingssysteem.
2. Start de installatie. Je kunt alle standaardinstellingen laten staan. Op Windows wordt XAMPP geïnstalleerd in de map **C:\xampp**.
3. Krijg je een melding van Windows Firewall? Kies dan *Toegang toestaan* (alleen voor privénetwerken is genoeg).
4. Open na de installatie het **XAMPP Control Panel**.
5. Klik bij **Apache** en bij **MySQL** op **Start**. Als alles goed gaat, worden ze groen.

Open nu je browser en ga naar **http://localhost**. Zie je de welkomspagina van XAMPP? Dan werkt je webserver!

#### ⚠ Let op

Wordt Apache niet groen, maar krijg je een foutmelding over **poort 80**? Dan gebruikt een ander programma die poort al, bijvoorbeeld Skype of een ander serverprogramma. Sluit dat programma, of vraag je docent om hulp. Laat het XAMPP Control Panel open terwijl je werkt; als je Apache of MySQL stopt, werkt je API niet meer.

## De map htdocs

Alle bestanden die Apache mag laten zien, zet je in de map **htdocs**. Op Windows is dat `C:\xampp\htdocs` en op een Mac `/Applications/XAMPP/htdocs`.

Een bestand in htdocs kun je in je browser openen via `http://localhost/`, gevolgd door het pad. Het bestand `C:\xampp\htdocs\api-test\hallo.php` open je dus via `http://localhost/api-test/hallo.php`.

## Visual Studio Code

Je code schrijf je in **Visual Studio Code**, kortweg **VS Code**. Download het gratis via [code.visualstudio.com](https://code.visualstudio.com) en installeer het. Installeer daarna in VS Code (via het icoontje met de vier blokjes, **Extensions**) de extensie **PHP Intelephense**. Die geeft je kleurtjes, suggesties en waarschuwingen terwijl je PHP typt.

## Je eerste mini-API

Tijd om iets te maken! Maak in htdocs een nieuwe map `api-test`. Open die map in VS Code (**File** → **Open Folder**) en maak een bestand `hallo.php`:

```
api-test/hallo.php

<?php
// Vertel de client dat we JSON terugsturen
header('Content-Type: application/json');

// De gegevens die we willen teruggeven
$antwoord = [
    'bericht' => 'Hallo! Mijn eerste API werkt.',
    'tijd'    => date('H:i:s'),
];

// Zet de PHP-array om naar JSON en stuur hem terug
echo json_encode($antwoord);
```

Open `http://localhost/api-test/hallo.php` in je browser. Je ziet nu geen opgemaakte webpagina, maar gegevens:

```
Resultaat

{"bericht":"Hallo! Mijn eerste API werkt.,"tijd":"14:05:12"}
```

Gefeliciteerd, dit is je eerste API! Hij doet nog niet veel, maar alle onderdelen zijn er al: een request komt binnen, PHP maakt een antwoord, en er gaat JSON terug. In de volgende hoofdstukken leer je wat `header()` en `json_encode()` precies doen.

## Postman

Met je browser kun je alleen **GET**-requests doen: gegevens ophalen. Voor een API wil je ook gegevens versturen, wijzigen en verwijderen. Daarvoor gebruik je **Postman**, een programma waarmee je zelf requests kunt maken en de response precies kunt bekijken.

1. Ga naar **postman.com** en download Postman.
2. Installeer het en open het. Postman vraagt je om een (gratis) account. Je kunt er een aanmaken, of kiezen om Postman zonder account te gebruiken als die mogelijkheid er is.
3. Klik op **New → HTTP** (of op het plusje bij de tabbladen) om een nieuw request te maken.
4. Laat de methode op **GET** staan, typ de URL `http://localhost/api-test/hallo.php` en klik op **Send**.

Onderaan zie je de response. Je ziet de **body** (je JSON), de **statuscode** (`200 OK`), hoe lang het duurde en hoe groot het antwoord was. Klik op het tabblad **Headers** in de response om de response-headers te bekijken.

### Goed om te weten

Er zijn ook andere programma's die hetzelfde doen als Postman, zoals **Insomnia** en **Bruno**. Voor dit boek maakt het niet uit welke je gebruikt; ze werken bijna hetzelfde. Wij gaan uit van Postman.

## De database klaarzetten

De forum-API gebruikt een database met de naam `forum_api_vb`. Je docent geeft je het bestand `forum_api_vb.sql`. Daarin staan de tabellen en voorbeeldgegevens. Zo zet je de database klaar:

1. Ga naar `http://localhost/phpmyadmin`.
2. Klik links op **Nieuw** (of *New*).
3. Typ bij *Databasenaam* `forum_api_vb` en kies als collatie `utf8mb4_general_ci`. Klik op **Aanmaken**.
4. Klik links op je nieuwe database en daarna bovenaan op het tabblad **Importeren**.
5. Kies bij *Bestand om te importeren* het bestand `forum_api_vb.sql` en klik onderaan op **Importeren** (of *Starten*).

Als alles goed is gegaan, zie je links vier tabellen: `replies`, `threads`, `topics` en `users`. Klik er eens op om de gegevens te bekijken.

Dit zijn de kolommen van de tabellen:

| Tabel                | Kolommen                                                                                                                                                        |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>threads</code> | <code>id</code> , <code>title</code> , <code>content</code> , <code>user_id</code> , <code>created_at</code> , <code>updated_at</code>                          |
| <code>topics</code>  | <code>id</code> , <code>title</code> , <code>content</code> , <code>thread_id</code> , <code>user_id</code> , <code>created_at</code> , <code>updated_at</code> |
| <code>replies</code> | <code>id</code> , <code>content</code> , <code>topic_id</code> , <code>user_id</code> , <code>created_at</code> , <code>updated_at</code>                       |
| <code>users</code>   | <code>id</code> , <code>username</code> , <code>email</code> , <code>password</code> , <code>avatar</code> , <code>created_at</code> , <code>updated_at</code>  |

Zie je de kolommen die op `_id` eindigen? Die verwijzen naar een rij in een andere tabel. `thread_id` in `topics` zegt bij welke thread een topic hoort. Zo'n kolom heet een **foreign key** (vreemde sleutel).

### ⚠ Let op

In XAMPP heeft de databasegebruiker `root` standaard **geen wachtwoord**. Dat is handig op je eigen computer, maar op een echte server mag dat nooit. Daar gebruik je altijd een aparte gebruiker met een sterk wachtwoord.

### 📌 Samenvatting

- **XAMPP** bevat Apache (webserver), MariaDB/MySQL (database), PHP en phpMyAdmin.
- Start Apache en MySQL in het **XAMPP Control Panel**.
- Je bestanden zet je in **htdocs**; je opent ze via `http://localhost/...`
- Met **Postman** kun je zelf requests maken met elke methode, en de response bekijken.
- De database `forum_api_vb` zet je klaar via **phpMyAdmin**.

### ✎ Opdracht: Hoofdstuk 3

1. Installeer XAMPP, VS Code en Postman. Start Apache en MySQL.
2. Maak `hallo.php` na en open het in je browser én in Postman. Wat is de statuscode? En welke waarde heeft de header `Content-Type` in de response?
3. Pas `hallo.php` aan, zodat het ook je naam en je lievelingsgetal teruggeeft.
4. Zet de database `forum_api_vb` klaar. Hoeveel rijen staan er in de tabel `threads`? En in `topics`?
5. In welke tabel en kolom kun je zien bij welke topic een reply hoort?

## 4. HTTP-methodes: wat wil je doen?

### Werkwoorden van het web

In hoofdstuk 2 zag je dat elk request begint met een **methode**, zoals **GET** of **POST**. De methode vertelt de server **wat** je wilt doen. Je kunt de methodes zien als de werkwoorden van het web: *ophalen*, *aanmaken*, *wijzigen*, *verwijderen*. De URL vertelt **waarmee** je dat wilt doen.

**GET** `/threads/1` betekent dus: *haal thread 1 op*. En **DELETE** `/threads/1` betekent: *verwijder thread 1*. Dezelfde URL, maar een heel andere betekenis.

### CRUD

Bijna alles wat een app met gegevens doet, valt in vier groepen. Samen heten ze **CRUD**:

| Letter   | Betekenis            | Voorbeeld                 | HTTP-methode               | SQL           |
|----------|----------------------|---------------------------|----------------------------|---------------|
| <b>C</b> | Create (aanmaken)    | Een nieuwe reply plaatsen | <b>POST</b>                | <b>INSERT</b> |
| <b>R</b> | Read (lezen)         | Alle threads bekijken     | <b>GET</b>                 | <b>SELECT</b> |
| <b>U</b> | Update (wijzigen)    | Een topic aanpassen       | <b>PUT</b> of <b>PATCH</b> | <b>UPDATE</b> |
| <b>D</b> | Delete (verwijderen) | Een reply weghalen        | <b>DELETE</b>              | <b>DELETE</b> |

### De methodes één voor één

#### GET: gegevens ophalen

**GET** gebruik je om gegevens **op te halen**. Een GET-request heeft geen body. Wil je extra opties meegeven, dan zet je die in de URL, in de query string.

##### GET-voorbeelden

```
GET /forum-api/threads      ← alle threads ophalen
GET /forum-api/threads/5    ← alleen thread 5 ophalen
```

Als je een URL in je browser typt, doet de browser altijd een GET-request.

#### POST: iets nieuws aanmaken

**POST** gebruik je om iets **nieuws aan te maken**. De gegevens van het nieuwe ding stuur je mee in de body. De server bepaalt zelf het **id** van het nieuwe ding.

##### POST-voorbeeld

```
POST /forum-api/replies
Content-Type: application/json

{"content": "Goed idee!", "topic_id": 2, "user_id": 3}
```

## PUT: iets helemaal vervangen

**PUT** gebruik je om een bestaand ding **helemaal te vervangen**. Je stuurt alle velden mee, ook de velden die niet veranderen. Wat je niet meestuurt, zou eigenlijk leeg moeten worden. Daarom zijn bij PUT in onze API alle velden verplicht.

### PUT-voorbeeld

```
PUT /forum-api/threads/5
Content-Type: application/json

{"title": "Nieuwe titel", "content": "Nieuwe tekst", "user_id": 1}
```

## PATCH: iets gedeeltelijk wijzigen

**PATCH** gebruik je om een bestaand ding **gedeeltelijk te wijzigen**. Je stuurt alleen de velden mee die veranderen. De rest blijft zoals het was.

### PATCH-voorbeeld

```
PATCH /forum-api/threads/5
Content-Type: application/json

{"title": "Alleen de titel is anders"}
```

### Goed om te weten

Het verschil tussen PUT en PATCH kun je vergelijken met je kamer. Bij PUT haal je alles eruit en zet je er complete nieuwe inrichting in. Bij PATCH hang je alleen een nieuwe poster op; de rest blijft staan.

## DELETE: iets verwijderen

**DELETE** gebruik je om iets te **verwijderen**. Er is geen body nodig: de URL zegt al wat er weg moet.

### DELETE-voorbeeld

```
DELETE /forum-api/replies/107
```

## OPTIONS: wat mag er?

**OPTIONS** is een bijzondere methode. Je vraagt ermee aan de server: *welke methodes en headers mag ik bij deze URL gebruiken?* Je gebruikt OPTIONS zelf bijna nooit. Maar je browser doet het automatisch als een website een API op een ander adres wil gebruiken. Zo'n automatisch OPTIONS-request heet een **preflight**. Daarover leer je alles in hoofdstuk 12, bij CORS.

### Goed om te weten

Er bestaan nog een paar methodes, zoals **HEAD**. Die werkt als GET, maar de server stuurt alleen de headers terug en geen body. Voor onze API heb je die niet nodig.

## Veilig en idempotent

Twee moeilijke woorden die je vaak tegenkomt bij methodes:

- Een methode is **veilig** (*safe*) als hij niets verandert op de server. GET en OPTIONS zijn veilig: je kunt ze zo vaak doen als je wilt, er gaat niets kapot.
- Een methode is **idempotent** als het niet uitmaakt of je hem één keer of tien keer doet: het resultaat is hetzelfde. Verwijder je thread 5 tien keer, dan is thread 5 gewoon weg. Zet je met PUT tien keer dezelfde titel, dan staat er gewoon die titel. **POST is niet idempotent**: tien keer POST betekent tien nieuwe replies!

| Methode | Doel                  | Body? | Veilig? | Idempotent?  |
|---------|-----------------------|-------|---------|--------------|
| GET     | Ophalen               | Nee   | Ja      | Ja           |
| POST    | Aanmaken              | Ja    | Nee     | Nee          |
| PUT     | Helemaal vervangen    | Ja    | Nee     | Ja           |
| PATCH   | Gedeeltelijk wijzigen | Ja    | Nee     | Meestal niet |
| DELETE  | Verwijderen           | Nee   | Nee     | Ja           |
| OPTIONS | Vragen wat mag        | Nee   | Ja      | Ja           |

Waarom is dit belangrijk? Stel dat je internet even hapert en je app weet niet of het request is aangekomen. Bij een idempotente methode kan de app het request veilig opnieuw sturen. Bij POST niet, want dan krijg je misschien twee dezelfde replies.

## De methode uitlezen in PHP

In PHP lees je de methode van het request uit met `$_SERVER['REQUEST_METHOD']`. Maak in je map `api-test` het bestand `methode.php`:

 api-test/methode.php

```
<?php
header('Content-Type: application/json');

$methode = $_SERVER['REQUEST_METHOD'];

switch ($methode) {
    case 'GET':
        $bericht = 'Je wilt iets ophalen.';
        break;
    case 'POST':
        $bericht = 'Je wilt iets aanmaken.';
        break;
    case 'PUT':
    case 'PATCH':
        $bericht = 'Je wilt iets wijzigen.';
        break;
    case 'DELETE':
        $bericht = 'Je wilt iets verwijderen.';
        break;
    default:
        $bericht = 'Deze methode ken ik niet.';
}

echo json_encode([
    'methode' => $methode,
    'bericht' => $bericht,
]);
```

Test het in Postman: kies links van de URL steeds een andere methode en klik op **Send**. Je ziet dat hetzelfde PHP-bestand een ander antwoord geeft, afhankelijk van de methode. Precies zo werkt onze forum-API straks ook.



### Samenvatting

- De **methode** zegt wat je wilt doen, de **URL** zegt waarmee.
- **CRUD**: Create = **POST**, Read = **GET**, Update = **PUT/PATCH**, Delete = **DELETE**.
- **PUT** vervangt alles, **PATCH** wijzigd alleen de meegestuurde velden.
- **OPTIONS** vraagt wat er mag; de browser gebruikt het voor een **preflight**.
- **Veilig** = verandert niets; **idempotent** = vaker doen geeft hetzelfde resultaat.
- In PHP lees je de methode uit met `$_SERVER['REQUEST_METHOD']`.



### Opdracht: Hoofdstuk 4

1. Welke HTTP-methode hoort bij elke actie? a. Een nieuwe gebruiker registreren. b. De lijst met topics bekijken. c. Alleen je profielfoto veranderen. d. Een reply verwijderen. e. Een thread helemaal opnieuw invullen.
2. Leg het verschil uit tussen PUT en PATCH. Gebruik een eigen voorbeeld.
3. Waarom is POST niet idempotent en DELETE wel?
4. Maak `methode.php` na en test alle methodes in Postman. Wat krijg je terug bij de methode **OPTIONS**?
5. Pas `methode.php` aan: bij de methode **GET** moet er ook een lijstje met drie (verzonnen) threads worden teruggestuurd.

## 5. Headers: extra informatie bij requests en responses

### Wat zijn headers?

Als je een pakketje verstuurt, schrijf je er meer op dan alleen het adres. Misschien staat er *breekbaar* op, of *niet in de brievenbus*. Die extra informatie gaat niet over de inhoud, maar helpt de postbode wel.

**Headers** zijn precies dat: extra informatie bij een request of response. Elke header is een regel met een **naam**, een dubbele punt en een **waarde**:

```

Headers
Content-Type: application/json
Accept: application/json
  
```

Hoofdletters maken bij de naam van een header niet uit: `content-type` en `Content-Type` zijn hetzelfde.

### Belangrijke request-headers

Deze headers stuurt de **client** mee:

| Header                      | Voorbeeld                          | Betekenis                                                   |
|-----------------------------|------------------------------------|-------------------------------------------------------------|
| <code>Host</code>           | <code>localhost</code>             | Naar welke server het request gaat. Verplicht.              |
| <code>Accept</code>         | <code>application/json</code>      | Welk formaat de client graag terug wil hebben.              |
| <code>Content-Type</code>   | <code>application/json</code>      | In welk formaat de body van het request is.                 |
| <code>Content-Length</code> | <code>74</code>                    | Hoeveel bytes de body is.                                   |
| <code>Authorization</code>  | <code>Bearer abc123...</code>      | Bewijs wie je bent, bijvoorbeeld een token.                 |
| <code>User-Agent</code>     | <code>PostmanRuntime/7.39.0</code> | Welk programma het request stuurt.                          |
| <code>Origin</code>         | <code>http://localhost:5500</code> | Vanaf welke website een request komt. Belangrijk voor CORS. |

## Belangrijke response-headers

Deze headers stuurt de **server** terug:

| Header                      | Voorbeeld                             | Betekenis                                                              |
|-----------------------------|---------------------------------------|------------------------------------------------------------------------|
| Content-Type                | application/json;<br>charset=utf-8    | In welk formaat de body van de response is.                            |
| Content-Length              | 785                                   | Hoeveel bytes de body is.                                              |
| Location                    | http://localhost/forum-api/threads/23 | Waar een nieuw aangemaakt ding te vinden is (bij <b>201 Created</b> ). |
| Allow                       | GET, POST                             | Welke methodes wél mogen (bij <b>405 Method Not Allowed</b> ).         |
| Access-Control-Allow-Origin | *                                     | Welke websites de API mogen gebruiken (CORS).                          |
| Cache-Control               | no-cache                              | Of en hoe lang de response bewaard mag worden.                         |

## Content-Type: de belangrijkste header

Voor een API is **Content-Type** de allerbelangrijkste header. Hij vertelt wat voor soort gegevens er in de body staan. Zo'n soort heet een **MIME-type** of **media type**:

| Media type                        | Wat is het?                                                                |
|-----------------------------------|----------------------------------------------------------------------------|
| application/json                  | JSON: het standaardformaat voor API's.                                     |
| text/html                         | Een HTML-pagina. Dit is wat PHP standaard stuurt.                          |
| text/plain                        | Gewone tekst.                                                              |
| application/x-www-form-urlencoded | Gegevens uit een gewoon HTML-formulier: <b>title=Hallo&amp;user_id=1</b> . |
| multipart/form-data               | Een formulier met bestanden erin (bijvoorbeeld een foto uploaden).         |

### ⚠ Let op

PHP stuurt standaard **Content-Type: text/html**. Vergeet je in een API de header **Content-Type: application/json**, dan denkt de client dat hij HTML krijgt. Sommige programma's kunnen de JSON dan niet goed lezen. Zet de header dus altijd!

## Headers versturen in PHP

Een response-header stuur je in PHP met de functie `header()`:

### Response-headers versturen

```
header('Content-Type: application/json; charset=utf-8');  
header('Location: http://localhost/forum-api/threads/23');  
header('X-API-Naam: forum_api'); // een zelfbedachte header
```

### Let op

Headers moeten **vóór** de body worden verstuurd. Heb je al iets ge-`echo`d, of staat er zelfs maar een spatie of lege regel vóór `<?php`, dan is het te laat. PHP geeft dan de waarschuwing *Cannot modify header information - headers already sent*. Zorg dus dat `<?php` echt helemaal bovenaan je bestand staat.

## Request-headers uitlezen in PHP

De headers van het request lees je uit via `$_SERVER`. PHP zet elke header daarin met `HTTP_` ervoor, in hoofdletters, met underscores in plaats van streepjes. De header `User-Agent` vind je dus in `$_SERVER['HTTP_USER_AGENT']`. Twee headers zijn een uitzondering: `Content-Type` en `Content-Length` staan in `$_SERVER['CONTENT_TYPE']` en `$_SERVER['CONTENT_LENGTH']`, zonder `HTTP_`.

Onder Apache (dus ook in XAMPP) kun je ook de functie `getallheaders()` gebruiken. Die geeft alle request-headers in één array. Maak `headers.php`:

api-test/headers.php

```
<?php
header('Content-Type: application/json; charset=utf-8');
header('X-Api-Naam: api-test');

echo json_encode([
    'jouw_programma' => $_SERVER['HTTP_USER_AGENT'] ?? 'onbekend',
    'jij_wilt_graag' => $_SERVER['HTTP_ACCEPT'] ?? 'niets opgegeven',
    'alle_headers'    => getallheaders(),
], JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES);
```

Open het in Postman en kijk wat je terugkrijgt. Klik daarna in Postman bij je request op het tabblad **Headers** en voeg een eigen header toe, bijvoorbeeld `X-Klas` met als waarde `4B`. Stuur het request opnieuw. Zie je je eigen header terug in `alle_headers`? Kijk ook in de response-headers: daar staat nu `X-Api-Naam`.

### Tip

De `??` heet de **null coalescing operator**. `$_SERVER['HTTP_ACCEPT'] ?? 'niets opgegeven'` betekent: gebruik `$_SERVER['HTTP_ACCEPT']` als die bestaat, en anders de tekst `'niets opgegeven'`. Zo voorkom je een waarschuwing als een header niet is meegestuurd.

### Samenvatting

- **Headers** zijn extra informatie bij een request of response: **Naam: waarde**.
- Belangrijke request-headers: `Host`, `Accept`, `Content-Type`, `Authorization`, `Origin`.
- Belangrijke response-headers: `Content-Type`, `Location`, `Allow`, `Access-Control-Allow-Origin`.
- `Content-Type` vertelt het formaat van de body. Voor API's is dat `application/json`.
- In PHP stuur je een header met `header()`, **voordat** je iets `echo`t.
- Request-headers lees je uit met `$_SERVER['HTTP_...']` of `getallheaders()`.

**Opdracht: Hoofdstuk 5**

1. Wat is het verschil tussen de request-header **Accept** en de request-header **Content-Type**?
2. In welke variabele vind je in PHP de request-header **Authorization**?
3. Maak **headers.php** na. Voeg in Postman de header **X-Klas** toe en controleer of je hem terugziet.
4. Wat gaat er mis in deze code? Leg uit en verbeter het.

**Opdracht 5.4**

```
<?php
echo 'Hallo';
header('Content-Type: application/json');
```

5. Pas **hallo.php** uit hoofdstuk 3 aan: laat de API alleen JSON teruggeven als de client in de header **Accept** om **application/json** vraagt. Anders stuurt hij gewone tekst terug (**Content-Type: text/plain**) met de zin *Ik spreek alleen JSON*. Tip: gebruik **str\_contains()**.

## 6. JSON: de taal van API's

### Wat is JSON?

Als twee programma's gegevens uitwisselen, moeten ze allebei begrijpen hoe die gegevens zijn opgeschreven. Het meest gebruikte formaat daarvoor is **JSON** (spreek uit: *djee-son*). Het staat voor **JavaScript Object Notation**. JSON komt oorspronkelijk uit JavaScript, maar bijna elke programmeertaal kan het lezen en schrijven, ook PHP.

JSON is gewoon tekst. Daardoor kun je het makkelijk versturen via HTTP, en kun je het als mens ook nog lezen:

#### Een thread in JSON

```
{
  "id": 1,
  "title": "Huiswerkhulp",
  "content": "Stel hier je vragen!",
  "user_id": 11,
  "is_gesloten": false,
  "tags": ["school", "hulp"],
  "updated_at": null
}
```

### De regels van JSON

JSON kent maar een paar bouwstenen:

| Soort   | Voorbeeld                               | Uitleg                                                       |
|---------|-----------------------------------------|--------------------------------------------------------------|
| Object  | <code>{ "naam": "Sam" }</code>          | Tussen accolades. Paren van een naam en een waarde.          |
| Array   | <code>[1, 2, 3]</code>                  | Tussen blokhaken. Een lijst met waarden.                     |
| String  | <code>"Hallo"</code>                    | Tekst, altijd tussen <b>dubbele</b> aanhalingstekens.        |
| Number  | <code>42</code> of <code>19.99</code>   | Een getal, zonder aanhalingstekens. Een punt voor decimalen. |
| Boolean | <code>true</code> of <code>false</code> | Waar of niet waar, in kleine letters.                        |
| Null    | <code>null</code>                       | Leeg: er is geen waarde.                                     |

Objecten en arrays mag je in elkaar stoppen. Zo kun je een lijst met threads maken, waarbij elke thread een object is:

### Een lijst met objecten

```
[
  { "id": 1, "title": "Games" },
  { "id": 2, "title": "Huiswerkhulp" }
]
```

### ⚠ Let op

JSON is heel streng. Deze fouten maken beginners vaak:

- Enkele aanhalingstekens gebruiken: `{ 'naam': 'Sam' }` is **fout**. Het moet `{ "naam": "Sam" }` zijn.
- Een komma na het laatste item: `[1, 2, 3, ]` is **fout**.
- De naam niet tussen aanhalingstekens: `{ naam: "Sam" }` is **fout**.
- Commentaar toevoegen met `//`: dat mag in JSON **niet**.

## JSON in PHP

PHP heeft twee functies om met JSON te werken:

- `json_encode()` zet een PHP-waarde (meestal een array) om naar een JSON-tekst.
- `json_decode()` zet een JSON-tekst om naar een PHP-waarde.

Een PHP-array met sleutels die tekst zijn (een **associatieve array**) wordt een JSON-**object**. Een array met de sleutels 0, 1, 2, ... (een **lijst**) wordt een JSON-**array**.

### Van PHP naar JSON

```
$thread = [
    'id'    => 1,
    'title' => 'Crème brûlée maken',
    'tags'  => ['koken', 'toetje'],
];

echo json_encode($thread);
// {"id":1,"title":"Cr\00e8me br\00fbl\00e9e maken","tags":["koken","toetje"]}
```

```
echo json_encode($thread, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
// {
//     "id": 1,
//     "title": "Crème brûlée maken",
//     "tags": [
//         "koken",
//         "toetje"
//     ]
// }
```

Met de tweede parameter van `json_encode()` geef je **flags** (opties) mee. Meerdere flags combineer je met een streepje `|`:

| Flag                                | Wat doet het?                                                                |
|-------------------------------------|------------------------------------------------------------------------------|
| <code>JSON_PRETTY_PRINT</code>      | Netjes over meerdere regels, met inspringen. Makkelijker te lezen.           |
| <code>JSON_UNESCAPED_UNICODE</code> | Letters zoals é en ü gewoon laten staan, in plaats van <code>\u00e9</code> . |
| <code>JSON_UNESCAPED_SLASHES</code> | Een <code>/</code> niet veranderen in <code>\\</code> . Handig voor URL's.   |

Andersom, van JSON naar PHP, gebruik je `json_decode()`. Geef als tweede parameter `true` mee; dan krijg je een PHP-array terug:

#### Van JSON naar PHP

```
$json = '{"title": "Huiswerkhulp", "user_id": 1}';

$data = json_decode($json, true);

echo $data['title']; // Huiswerkhulp
```

Is de JSON niet geldig, dan geeft `json_decode()` de waarde `null` terug. Daar moet je in een API altijd op controleren.

## JSON ontvangen in een API

Als een client JSON in de body van een request stuurt, zet PHP dat **niet** in `$_POST`. `$_POST` werkt alleen voor gewone formulierdata. De ruwe body lees je uit met `file_get_contents('php://input')`. Maak `echo.php`, een API die de ontvangen JSON gewoon terugstuurt:

#### api-test/echo.php

```
<?php
header('Content-Type: application/json; charset=utf-8');

// De ruwe body van het request uitlezen
$body = file_get_contents('php://input');

// Omzetten naar een PHP-array
$data = json_decode($body, true);

if ($data === null) {
    echo json_encode(['fout' => 'Dit is geen geldige JSON.']);
    exit;
}

echo json_encode([
    'ontvangen' => $data,
    'aantal_velden' => count($data),
], JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
```

Test het in Postman:

1. Kies de methode **POST** en de URL `http://localhost/api-test/echo.php`.
2. Klik onder de URL op het tabblad **Body**, kies **raw** en rechts in het uitklapmenu **JSON**.
3. Typ in het tekstvak: `{"naam": "Sam", "klas": "4B"}` en klik op **Send**.

Postman zet de header `Content-Type: application/json` dan automatisch voor je. Probeer daarna eens kapotte JSON te sturen, bijvoorbeeld zonder de laatste accolade.

### Samenvatting

- **JSON** is een tekstformaat om gegevens op te schrijven, met objecten `{}`, arrays `[]`, strings, numbers, booleans en `null`.
- JSON is streng: dubbele aanhalingstekens, geen komma na het laatste item, geen commentaar.
- `json_encode()` maakt van PHP JSON; `json_decode($json, true)` maakt van JSON een PHP-array.
- Handige flags: `JSON_PRETTY_PRINT`, `JSON_UNESCAPED_UNICODE`, `JSON_UNESCAPED_SLASHES`.
- JSON uit de body lees je met `file_get_contents('php://input')`, niet met `$_POST`.

### Opdracht: Hoofdstuk 6

1. Er zitten drie fouten in deze JSON. Welke? `{'titel': "Games", aantal: 3, "tags": ["pc", "console"], }`
2. Schrijf in JSON een object voor een reply met een `id`, een `content`, een `topic_id`, een `user_id` en een lege `updated_at`.
3. Wat is het verschil in JSON tussen de PHP-arrays `['a', 'b']` en `['x' => 'a', 'y' => 'b']`?
4. Maak `echo.php` na en test het in Postman met geldige en ongeldige JSON.
5. Waarom werkt `$_POST` niet als een client JSON stuurt?

## 7. HTTP-responses en statuscodes

### Hoe ging het?

Bij elke response stuurt de server een **statuscode** mee: een getal van drie cijfers dat in één keer vertelt hoe het ging. Een client kan daardoor meteen zien of het gelukt is, zonder de body te hoeven lezen. Een app kan bijvoorbeeld bij **404** de melding *Niet gevonden* laten zien, en bij **500** *Er ging iets mis, probeer het later opnieuw*.

### De vijf groepen

Het eerste cijfer van een statuscode vertelt tot welke groep hij hoort:

| Groep      | Betekenis          | Kort gezegd                                                    |
|------------|--------------------|----------------------------------------------------------------|
| <b>1xx</b> | Informatie         | <i>Even wachten, ik ben bezig. (Kom je bijna nooit tegen.)</i> |
| <b>2xx</b> | Gelukt             | <i>Alles in orde!</i>                                          |
| <b>3xx</b> | Doorverwijzing     | <i>Wat je zoekt, staat ergens anders.</i>                      |
| <b>4xx</b> | Fout van de client | <i>Jij deed iets fout.</i>                                     |
| <b>5xx</b> | Fout van de server | <i>Ik (de server) deed iets fout.</i>                          |

Het verschil tussen 4xx en 5xx is heel belangrijk. Bij een **4xx** moet de client iets anders doen: een andere URL gebruiken, andere gegevens sturen of inloggen. Opnieuw proberen heeft geen zin. Bij een **5xx** ligt het aan de server; misschien werkt het later wel.

### De belangrijkste statuscodes

#### Gelukt (2xx)

| Code       | Naam       | Wanneer gebruik je hem?                                                                                            |
|------------|------------|--------------------------------------------------------------------------------------------------------------------|
| <b>200</b> | OK         | Het request is gelukt. Voor GET, PUT en PATCH, met de gegevens in de body.                                         |
| <b>201</b> | Created    | Er is iets nieuws aangemaakt, na een POST. Stuur ook de header <b>Location</b> mee met de URL van het nieuwe ding. |
| <b>204</b> | No Content | Gelukt, maar er is niets om terug te sturen. Vaak na een DELETE. Er mag dan <b>geen body</b> zijn.                 |

**Fout van de client (4xx)**

| Code | Naam                   | Wanneer gebruik je hem?                                                                                                        |
|------|------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 400  | Bad Request            | Het request is kapot of onbegrijpelijk. Bijvoorbeeld ongeldige JSON, of een id dat geen getal is.                              |
| 401  | Unauthorized           | Je bent niet ingelogd, of je inloggegevens kloppen niet. De server weet niet wie je bent.                                      |
| 403  | Forbidden              | De server weet wie je bent, maar je mag dit niet. Bijvoorbeeld een reply van iemand anders verwijderen.                        |
| 404  | Not Found              | Wat je zoekt, bestaat niet. De route bestaat niet, of thread 999 bestaat niet.                                                 |
| 405  | Method Not Allowed     | De URL bestaat, maar niet met deze methode. Bijvoorbeeld <b>DELETE</b> /threads (zonder id). Stuur de header <b>Allow</b> mee. |
| 409  | Conflict               | Het request botst met wat er al is. Bijvoorbeeld een gebruikersnaam die al bestaat.                                            |
| 415  | Unsupported Media Type | De client stuurt een formaat dat de server niet begrijpt, bijvoorbeeld XML.                                                    |
| 422  | Unprocessable Content  | Het request is goed opgebouwd, maar de inhoud klopt niet. Bijvoorbeeld een verplicht veld dat leeg is.                         |
| 429  | Too Many Requests      | De client stuurt te veel requests in korte tijd.                                                                               |

**Fout van de server (5xx)**

| Code | Naam                  | Wanneer gebruik je hem?                                               |
|------|-----------------------|-----------------------------------------------------------------------|
| 500  | Internal Server Error | Er ging iets onverwachts mis in je code of de database.               |
| 501  | Not Implemented       | Deze functie is (nog) niet gebouwd.                                   |
| 503  | Service Unavailable   | De server is tijdelijk niet beschikbaar, bijvoorbeeld door onderhoud. |

## Best practices voor foutcodes

Welke code je kiest, maakt echt uit. Een client-programmeur bouwt zijn app op jouw statuscodes. Houd je daarom aan deze afspraken:

**1. Stuur nooit `200 OK` bij een fout.** Soms zie je API's die altijd `200` sturen, met in de body `"error": "Niet gevonden"`. Dat is verwarrend: de statuscode zegt *gelukt*, de body zegt *mislukt*. De client moet dan elke body gaan lezen om te weten of het goed ging.

**2. Wees zo precies mogelijk.** Gebruik niet voor elke fout `400`. Is iets niet gevonden, stuur dan `404`. Klopt de invoer niet, stuur dan `422`.

**3. Ken de verschillen die vaak verwisseld worden:**

- **400 of 422?** `400` als het request zelf kapot is (ongeldige JSON). `422` als het request goed leesbaar is, maar de gegevens niet kloppen (titel is leeg).
- **401 of 403?** `401` als de server niet weet wie je bent. `403` als de server wel weet wie je bent, maar je het niet mag.
- **404 of 405?** `404` als de URL helemaal niet bestaat. `405` als de URL wel bestaat, maar niet met deze methode.

**4. Verzin geen eigen statuscodes.** Codes als `210` of `211` bestaan niet officieel. Clients en tools begrijpen ze niet. Gebruik altijd de officiële codes.

**5. Vertel bij een fout wat er mis is, maar verraad geen geheimen.** Geef een duidelijke melding (*Het veld title is verplicht*), zodat de client weet wat hij moet verbeteren. Maar laat bij een `500` nooit de echte PHP- of databasefout zien. Daaruit kan een hacker afleiden hoe je systeem werkt. Schrijf de echte fout in een logbestand.

**6. Kies de code die past bij de situatie, niet bij je code.** Denk vanuit de client: wat moet de client nu doen? Opnieuw proberen, iets verbeteren, inloggen of opgeven?

## Statuscodes in PHP

In PHP stel je de statuscode in met `http_response_code()`. Maak `status.php`:

```
api-test/status.php

<?php
header('Content-Type: application/json; charset=utf-8');

$id = $_GET['id'] ?? null;

// Geen id meegegeven: het request is niet compleet
if ($id === null) {
    http_response_code(400);
    echo json_encode(['fout' => 'Geef een id mee, bijvoorbeeld ?id=1']);
    exit;
}

// Alleen id 1 bestaat in dit voorbeeld
if ($id != 1) {
    http_response_code(404);
    echo json_encode(['fout' => "Thread $id bestaat niet."]);
    exit;
}

http_response_code(200);
echo json_encode(['id' => 1, 'title' => 'Games']);
```

Test het in Postman met `status.php`, `status.php?id=1` en `status.php?id=7`, en kijk steeds naar de statuscode rechtsboven in de response.

### Tip

Doe je niets, dan stuurt PHP automatisch `200 OK`. En `exit` stopt het script meteen. Dat is handig na een fout: dan wordt de rest van je code niet meer uitgevoerd.

### Samenvatting

- Een **statuscode** zegt in één getal hoe het request ging.
- **2xx** = gelukt, **3xx** = doorverwijzing, **4xx** = fout van de client, **5xx** = fout van de server.
- Belangrijkste codes: `200`, `201`, `204`, `400`, `401`, `403`, `404`, `405`, `422`, `500`.
- Stuur nooit `200` bij een fout, verzin geen eigen codes en verraad bij een `500` geen details.
- In PHP stel je de statuscode in met `http_response_code()`.

**Opdracht: Hoofdstuk 7**

1. Wat is het verschil tussen een 4xx- en een 5xx-statuscode? Wat moet de client in beide gevallen doen?
2. Welke statuscode hoort bij deze situaties? a. Een nieuwe reply is aangemaakt. b. `GET /replies/5000`, maar die reply bestaat niet. c. Je stuurt `{"title": }` als body. d. Je stuurt een geldige thread, maar zonder titel. e. Je wilt een thread van iemand anders verwijderen. f. De database is uitgevallen. g. Je stuurt `DELETE /threads` zonder id. h. Een thread is verwijderd en er is niets om terug te sturen.
3. Een API stuurt bij een fout deze response: statuscode `200`, body `{"success": false, "error": "Niet gevonden"}`. Wat is hier mis en hoe zou het beter moeten?
4. Waarom mag je bij een `500` de echte foutmelding van de database niet in de response zetten?
5. Maak `status.php` na. Voeg een controle toe: als het id geen getal is (bijvoorbeeld `?id=abc`), stuur je `400`. Tip: gebruik `ctype_digit()`.

## 8. RESTful API's

### Wat is REST?

Je kunt een API op heel veel manieren ontwerpen. Om te zorgen dat API's een beetje op elkaar lijken, zijn er afspraken bedacht. De bekendste set afspraken heet **REST**: *Representational State Transfer*. Een API die zich aan die afspraken houdt, heet een **RESTful API** of **REST API**.

Het voordeel: als je één REST API kent, snap je de meeste andere ook. Een programmeur die jouw API gebruikt, kan vaak al raden hoe de URL's eruitzien.

### De belangrijkste principes

- 1. Alles is een resource.** Een **resource** is een *ding* waar je API over gaat: een thread, een topic, een reply, een user. Elke resource heeft een eigen URL.
- 2. De URL is een zelfstandig naamwoord, de methode is het werkwoord.** In de URL staat **wat** (de resource), in de methode staat **wat je ermee doet**. Dus niet `/getThreads` of `/deleteThread/5`, maar `GET /threads` en `DELETE /threads/5`.
- 3. Client en server zijn gescheiden.** De server weet niet of de client een website, een app of Postman is. Zolang de afspraken hetzelfde blijven, kunnen ze allebei los van elkaar veranderen.
- 4. Stateless.** Elk request bevat alle informatie die nodig is. De server onthoudt niets tussen twee requests. (Dat kende je al van HTTP.)
- 5. Representaties.** Je krijgt nooit de resource zelf (de rij in de database), maar een **representatie** ervan: een beschrijving in een bepaald formaat. Bij ons is dat JSON.
- 6. Gebruik de standaarden van HTTP.** De juiste methodes, de juiste statuscodes en de juiste headers. Alles wat je in de vorige hoofdstukken hebt geleerd, komt hier samen.

### URL's ontwerpen: de regels

| Regel                                                        | Goed                                        | Fout                                             |
|--------------------------------------------------------------|---------------------------------------------|--------------------------------------------------|
| Gebruik zelfstandige naamwoorden, geen werkwoorden.          | <code>GET /threads</code>                   | <code>GET /getAllThreads</code>                  |
| Gebruik het <b>meervoud</b> voor een verzameling.            | <code>/threads/5</code>                     | <code>/thread/5</code>                           |
| Zet het id in het pad.                                       | <code>/threads/5</code>                     | <code>/threads?id=5</code>                       |
| Gebruik kleine letters en streepjes.                         | <code>/forum-posts</code>                   | <code>/ForumPosts</code>                         |
| Gebruik de query string voor filteren, sorteren en pagina's. | <code>/threads?sort=nieuw&amp;page=2</code> | <code>/threads/sorteer-nieuw/pagina-2</code>     |
| Laat relaties zien met nesting (niet te diep).               | <code>/threads/1/topics</code>              | <code>/threads/1/topics/4/replies/9/users</code> |

**i Goed om te weten**

Waarom het meervoud? Denk aan de URL als een map. `/threads` is de map met *alle* threads; `/threads/5` is thread nummer 5 *uit die map*. Daardoor gebruik je voor één en voor alle threads dezelfde naam. De originele versie van onze forum-API gebruikte `/thread/1` en `/threads`. In hoofdstuk 13 passen we dat aan.

## Een collectie en een item

Bij een REST API heb je voor elke resource meestal twee soorten URL's:

- Een **collectie**: alle items, zoals `/threads`.
- Een **item**: één ding, zoals `/threads/5`.

Samen met de methodes geeft dat een vast patroon. Dit zijn de endpoints van onze forum-API voor threads. Een **endpoint** is één combinatie van een methode en een URL.

| Methode | URL                        | Actie                            | Statuscode bij succes |
|---------|----------------------------|----------------------------------|-----------------------|
| GET     | <code>/threads</code>      | Alle threads ophalen             | 200 OK                |
| GET     | <code>/threads/{id}</code> | Eén thread ophalen               | 200 OK                |
| POST    | <code>/threads</code>      | Een nieuwe thread maken          | 201 Created           |
| PUT     | <code>/threads/{id}</code> | Een thread helemaal vervangen    | 200 OK                |
| PATCH   | <code>/threads/{id}</code> | Een thread gedeeltelijk wijzigen | 200 OK                |
| DELETE  | <code>/threads/{id}</code> | Een thread verwijderen           | 204 No Content        |

Voor topics en replies werkt het precies hetzelfde, met `/topics` en `/replies`. Het `{id}` is een plek waar het echte id komt te staan, zoals `5`.

Zie je dat POST naar de **collectie** gaat en niet naar een item? Logisch: het nieuwe ding heeft nog geen id. De server geeft het een id en vertelt in de `Location`-header waar je het kunt vinden.

## Versies

API's veranderen. Als je iets verandert waardoor oude apps niet meer werken, heet dat een **breaking change**. Om oude apps niet kapot te maken, geven veel API's een **versienummer** mee in de URL, bijvoorbeeld `/v1/threads`. Een nieuwe versie komt dan op `/v2/threads`, en oude apps kunnen `/v1` blijven gebruiken. In onze API zetten we het versienummer in de response (`api_version`), zodat de client kan zien met welke versie hij praat.

### Samenvatting

- **REST** is een set afspraken voor het ontwerpen van API's. Een API die zich eraan houdt, is **RESTful**.
- Alles is een **resource** met een eigen URL. De URL is het zelfstandig naamwoord, de methode het werkwoord.
- Gebruik het **meervoud**, zet het id in het pad en gebruik de query string voor filteren en sorteren.
- Per resource heb je een **collectie** (`/threads`) en **items** (`/threads/5`).
- Een **endpoint** is een combinatie van methode en URL.

### Opdracht: Hoofdstuk 8

1. Wat betekent het dat een API *RESTful* is? Noem drie principes.
2. Verbeter deze endpoints zodat ze RESTful zijn: a. `GET /getReplies` b. `POST /createTopic` c. `GET /thread?id=3` d. `POST /deleteReply/8` e. `GET /Users/12`
3. Ontwerp de endpoints (methode, URL, statuscode bij succes) voor een API van een bibliotheek met boeken. Je moet boeken kunnen bekijken, toevoegen, wijzigen en verwijderen.
4. Met welk endpoint zou je alle replies van topic 4 ophalen?
5. Waarom gaat een POST-request naar `/threads` en niet naar `/threads/23`?

## 9. Best practices voor response-structuren

### Waarom een vaste structuur?

Stel je voor dat elk endpoint van een API zijn antwoord op een andere manier opschrijft. Het ene endpoint stuurt een lijst, het andere een object met de gegevens onder `result`, en bij een fout staat de melding soms onder `error` en soms onder `message`. Een client-programmeur wordt daar gek van: voor elk endpoint moet hij andere code schrijven.

Daarom is de belangrijkste best practice: **gebruik overal dezelfde structuur**. Dan kan de client met één stukje code elke response lezen.

### De envelop: meta en data

Veel API's verpakken hun antwoord in een vaste **envelop**. Onze forum-API doet dat ook. Elke succesvolle response heeft twee delen:

- `meta`: informatie **over** de response, zoals de versie van de API en de statuscode.
- `data`: de gegevens zelf.

Bij een **lijst** ziet dat er zo uit. In `meta` staat met `count` ook hoeveel items er in de lijst zitten:

```
GET /threads

{
  "meta": {
    "api_version": "1.0",
    "api_name": "forum_api",
    "status": 200,
    "status_message": "OK",
    "count": 22
  },
  "data": [
    { "id": 1, "title": "Eum omnis unde quaerat et.", "...": "..." },
    { "id": 2, "title": "Et est reiciendis corporis.", "...": "..." }
  ]
}
```

Bij **één item** is `data` een object in plaats van een lijst:

```
GET /threads/1

{
  "meta": {
    "api_version": "1.0",
    "api_name": "forum_api",
    "status": 200,
    "status_message": "OK"
  },
  "data": {
    "id": 1,
    "title": "Eum omnis unde quaerat et.",
    "content": "Iusto quibusdam voluptates et assumenda enim ut.",
    "user_id": 11,
    "created_at": "2018-01-30 09:47:15",
    "updated_at": null
  }
}
```

## Fouten: een vaste foutstructuur

Ook fouten krijgen een vaste structuur. In plaats van `data` staat er dan `error`, met altijd een `message`. Bij fouten in de invoer staat er ook `details`, met per veld wat er mis is. Zo kan een app de melding precies onder het goede invoerveld zetten.

POST /threads met foute invoer (422)

```
{
  "meta": {
    "api_version": "1.0",
    "api_name": "forum_api",
    "status": 422,
    "status_message": "Unprocessable Content"
  },
  "error": {
    "message": "De invoer is niet geldig.",
    "details": {
      "title": "Het veld title is verplicht.",
      "user_id": "Het veld user_id moet een positief geheel getal zijn."
    }
  }
}
```

## De regels op een rij

- Altijd dezelfde vorm.** Elke response heeft `meta`, en daarnaast `data` (bij succes) of `error` (bij een fout). Nooit allebei, nooit geen van beide.
- `data` is een lijst bij een collectie en een object bij een item.** Ook als een lijst leeg is, stuur je een lege lijst `[]` en geen `null` of foutmelding. Een lege lijst is geen fout!
- Consistente namen.** Kies één schrijfwijze voor de namen van velden en houd je eraan. Onze API gebruikt `snake_case` (`user_id`, `created_at`), net als de kolommen in de database. Een andere veelgebruikte keuze is `camelCase` (`userId`), maar meng ze nooit door elkaar.
- Gebruik de juiste JSON-types.** Een getal is een number (`"user_id": 11`), geen string (`"user_id": "11"`). Waar of niet waar is `true/false`, geen `"ja"/"nee"` of `1/0`. Leeg is `null`.
- Een vast datumformaat.** Gebruik overal hetzelfde formaat voor datums en tijden. Onze API gebruikt het formaat uit de database: `"2018-01-30 09:47:15"`. Veel API's gebruiken de internationale standaard **ISO 8601**: `"2018-01-30T09:47:15Z"`.
- Stuur nooit gevoelige gegevens mee.** Een wachtwoord (ook een versleuteld wachtwoord!) hoort nooit in een response. Denk ook na over e-mailadressen en andere persoonlijke gegevens: moet de client die echt hebben?
- Geen body bij `204 No Content`.** Die statuscode betekent juist dat er niets terugkomt.
- Stuur na aanmaken of wijzigen het resultaat terug.** Na een POST, PUT of PATCH krijgt de client de nieuwe versie van het item terug, inclusief het `id` en de datums die de server heeft ingevuld. Dan hoeft de client niet nog een GET te doen.

 **Goed om te weten**

Bij heel grote lijsten stuur je niet alles in één keer terug, maar werk je met **pagina's** (*pagination*): bijvoorbeeld 20 items per keer. In **meta** staat dan ook op welke pagina je bent en hoeveel pagina's er zijn, en de client vraagt de volgende pagina op met **?page=2**. Onze forum-API is daar nog klein genoeg voor.

 **Samenvatting**

- De belangrijkste best practice: **overall dezelfde structuur**.
- Onze API gebruikt een envelop met **meta** en **data** (bij succes) of **meta** en **error** (bij een fout).
- **data** is een lijst bij een collectie en een object bij een item. Een lege lijst is geen fout.
- Gebruik consistente namen (*snake\_case*), de juiste JSON-types en een vast datumformaat.
- Stuur nooit wachtwoorden of andere gevoelige gegevens mee.

 **Opdracht: Hoofdstuk 9**

1. Waarom is het belangrijk dat alle responses van een API dezelfde structuur hebben?
2. Wat is er mis met deze response op **GET /replies/3**? Noem minstens vier dingen. **{"Reply": {"ID": "3", "Content": "Leuk!", "topicId": "2", "user\_id": 5, "wachtwoord\_schrijver": "371620aa75..."}, "status": "gelukt"}**
3. Stel dat onze API topics kan filteren op titel. Schrijf de complete response (volgens de structuur van onze API) voor **GET /topics?title=bestaatniet**, als er geen enkele topic gevonden wordt.
4. Schrijf de complete response voor een **PATCH /replies/3** waarbij de client **{"content": ""}** stuurt.
5. Waarom stuur je na een POST het nieuwe item terug in de body?

## 10. Routes en .htaccess

### Mooie URL's

In hoofdstuk 8 leerde je dat een REST API URL's heeft zoals `/threads/5`. Maar PHP werkt met bestanden. Als je `http://localhost/forum-api/threads/5` opent, zoekt Apache naar een map `threads` met daarin een bestand `5`. Dat bestaat niet, dus je krijgt een 404-pagina van Apache.

Zonder trucjes zou je URL er zo uit moeten zien:

#### Zonder rewriting

```
http://localhost/forum-api/index.php?resource=threads&id=5
```

Dat werkt, maar het is lelijk en niet RESTful. We willen dat de client `/threads/5` gebruikt, en dat Apache dat achter de schermen **herschrijft** naar `index.php?resource=threads&id=5`. Dat heet **URL rewriting**.

### Eén voordeur: de front controller

In onze API komt **elk** request binnen via één bestand: `index.php`. Het maakt niet uit of je `/threads`, `/topics/4` of `/replies` opvraagt. `index.php` kijkt welke URL en welke methode er gevraagd wordt, en stuurt het request door naar de juiste code. Zo'n bestand heet een **front controller**: de voordeur van je API.

#### Front controller

```
GET /forum-api/threads
GET /forum-api/threads/5
POST /forum-api/replies
```



```
→ .htaccess → index.php → juiste code
```

### Wat is .htaccess?

**.htaccess** is een instellingenbestand voor Apache. Je zet het in een map, en Apache gebruikt de instellingen voor die map en alle mappen eronder. De naam begint met een punt; daardoor is het op Mac en Linux een verborgen bestand.

Met **.htaccess** kun je veel dingen instellen. Wij gebruiken het voor URL rewriting, met de Apache-module **mod\_rewrite**. In XAMPP staat die module standaard aan.

#### ⚠ Let op

**.htaccess** werkt **alleen met Apache**. Gebruik je later een andere webserver, zoals nginx, dan moet je het rewrites op een andere manier instellen. Ook de ingebouwde webserver van PHP (`php -S`) leest geen **.htaccess**.

## Reguliere expressies in het kort

Om te zeggen welke URL's herschreven moeten worden, gebruikt mod\_rewrite **reguliere expressies** (*regex*): patronen om tekst mee te herkennen. Je hebt er maar een paar van nodig:

| Teken                 | Betekenis                               | Voorbeeld                                                     |
|-----------------------|-----------------------------------------|---------------------------------------------------------------|
| <code>^</code>        | Begin van de tekst                      | <code>^threads</code> = begint met <i>threads</i>             |
| <code>\$</code>       | Einde van de tekst                      | <code>threads\$</code> = eindigt met <i>threads</i>           |
| <code>[^/]</code>     | Elk teken, behalve een <code>/</code>   |                                                               |
| <code>+</code>        | Eén of meer keer het vorige             | <code>[^/]+</code> = één of meer tekens zonder <code>/</code> |
| <code>?</code>        | Het vorige mag er zijn, maar hoeft niet | <code>/?</code> = eventueel een <code>/</code>                |
| <code>[0-9]</code>    | Een cijfer                              | <code>[0-9]+</code> = een getal                               |
| <code>( )</code>      | Een groep: dit stuk wil ik onthouden    |                                                               |
| <code>\$1, \$2</code> | Wat in de eerste, tweede groep stond    |                                                               |

Het patroon `^([^\s]+)/([^\s]+)/?$` past dus bij een stuk zonder slash, een slash, nog een stuk zonder slash, en eventueel een slash aan het eind. Bij `threads/5` is `$1` gelijk aan `threads` en `$2` gelijk aan `5`.

## De .htaccess van onze API

Dit is de .htaccess van de forum-API. Hij staat in de hoofdmap van het project:

```
.htaccess

<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteBase /forum-api/

    # Deze mappen en bestanden mogen niet van buitenaf bereikbaar zijn
    RewriteRule ^(app|vendor)(/|$) - [F,L]
    RewriteRule ^composer\.(json|lock)$ - [F,L]

    # http://localhost/forum-api/threads/1
    RewriteCond %{REQUEST_FILENAME} !-f
    RewriteCond %{REQUEST_FILENAME} !-d
    RewriteRule ^([^\s]+)/([^\s]+)/?$ index.php?resource=$1&id=$2 [L,QSA]

    # http://localhost/forum-api/threads
    RewriteCond %{REQUEST_FILENAME} !-f
    RewriteCond %{REQUEST_FILENAME} !-d
    RewriteRule ^([^\s]+)/?$ index.php?resource=$1 [L,QSA]
</IfModule>
```

We lopen de regels door:

- `<IfModule mod_rewrite.c>` zorgt dat de regels alleen gebruikt worden als `mod_rewrite` aanstaat. Zo krijg je geen foutmelding als de module ontbreekt.
- `RewriteEngine On` zet het herschrijven aan.
- `RewriteBase /forum-api/` vertelt in welke map de API staat. Ons project staat in `htdocs/forum-api`, dus de basis is `/forum-api/`. Zet je je project in een andere map, dan moet je dit aanpassen!
- De twee regels onder *Deze mappen en bestanden...* **blokkeren** de mappen `app` en `vendor` en de bestanden van Composer. Het streepje `-` betekent *niets herschrijven*, en de flag `[F]` betekent *Forbidden*: Apache stuurt `403`. Niemand hoeft van buitenaf bij je broncode te kunnen.
- `RewriteCond %{REQUEST_FILENAME} !-f` is een **voorwaarde**: herschrijf alleen als het gevraagde **geen bestaand bestand** is (`!` = niet, `-f` = file). `!-d` betekent: **geen bestaande map** (`-d` = directory). Zo blijven echte bestanden, zoals een plaatje, gewoon werken.
- `RewriteRule` doet het echte werk: als de URL past bij het patroon, wordt hij herschreven naar `index.php` met de stukjes uit de URL als query string.

Achter een `RewriteRule` staan **flags** tussen blokhaken:

| Flag               | Betekenis                                                                                                                    |
|--------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>[L]</code>   | <i>Last</i> : dit is de laatste regel, stop hier met herschrijven.                                                           |
| <code>[QSA]</code> | <i>Query String Append</i> : plak de query string van de client erachter. Zonder QSA gaat <code>?sort=nieuw</code> verloren. |
| <code>[F]</code>   | <i>Forbidden</i> : stuur <code>403 Forbidden</code> .                                                                        |

### ⚠ Let op

Een `RewriteCond` geldt **alleen voor de eerstvolgende `RewriteRule`**. Daarom staan de twee voorwaarden in onze `.htaccess` twee keer: één keer voor elke regel. In de originele versie van dit project stonden ze er maar één keer, waardoor de tweede regel ook bestaande bestanden, zoals `index.php` zelf, zou kunnen herschrijven.

Zo worden de URL's herschreven:

| De client vraagt                                  | Apache maakt ervan                                                     |
|---------------------------------------------------|------------------------------------------------------------------------|
| <code>/forum-api/</code>                          | <code>index.php</code> (bestaande map, dus niet herschreven)           |
| <code>/forum-api/threads</code>                   | <code>index.php?resource=threads</code>                                |
| <code>/forum-api/threads/</code>                  | <code>index.php?resource=threads</code>                                |
| <code>/forum-api/threads/5</code>                 | <code>index.php?resource=threads&amp;id=5</code>                       |
| <code>/forum-api/threads/5?x=1</code>             | <code>index.php?resource=threads&amp;id=5&amp;x=1</code> (dankzij QSA) |
| <code>/forum-api/app/Database/Database.php</code> | <code>403 Forbidden</code>                                             |

In `index.php` kun je de stukjes dan gewoon uitlezen met `$_GET['resource']` en `$_GET['id']`.

## Routes: welke code bij welke URL?

De .htaccess stuurt alles naar `index.php`. Maar hoe weet `index.php` welke code hij moet uitvoeren?

Daarvoor gebruiken we een **routetabel**: een lijst die zegt welke code bij welke combinatie van methode en URL hoort. Een regel in die lijst heet een **route**.

Onze routetabel staat in `app/Routes/routes.php`. Het is een PHP-array met per methode een lijstje routes:

```

app/Routes/routes.php

<?php

use App\Http\Controllers\ThreadController;
use App\Http\Controllers\TopicController;
use App\Http\Controllers\ReplyController;

return [
    // Read
    'GET' => [
        // route => [Controller, 'methode']
        '/' => [ ThreadController::class, 'index' ],
        'threads' => [ ThreadController::class, 'index' ],
        'threads/{id}' => [ ThreadController::class, 'show' ],
        'topics' => [ TopicController::class, 'index' ],
        'topics/{id}' => [ TopicController::class, 'show' ],
        'replies' => [ ReplyController::class, 'index' ],
        'replies/{id}' => [ ReplyController::class, 'show' ],
    ],
    // Create
    'POST' => [
        'threads' => [ ThreadController::class, 'create' ],
        'topics' => [ TopicController::class, 'create' ],
        'replies' => [ ReplyController::class, 'create' ],
    ],
    // Update (volledig: alle velden verplicht)
    'PUT' => [
        'threads/{id}' => [ ThreadController::class, 'update' ],
        'topics/{id}' => [ TopicController::class, 'update' ],
        'replies/{id}' => [ ReplyController::class, 'update' ],
    ],
    // Update (gedeeltelijk: alleen meegestuurde velden)
    'PATCH' => [
        'threads/{id}' => [ ThreadController::class, 'patch' ],
        'topics/{id}' => [ TopicController::class, 'patch' ],
        'replies/{id}' => [ ReplyController::class, 'patch' ],
    ],
    // Delete
    'DELETE' => [
        'threads/{id}' => [ ThreadController::class, 'destroy' ],
        'topics/{id}' => [ TopicController::class, 'destroy' ],
        'replies/{id}' => [ ReplyController::class, 'destroy' ],
    ],
];

```

Lees een route zo: bij de methode `GET` en de URL `threads/{id}` hoort de methode `show` van de klasse `ThreadController`. Het `{id}` is een plek waar een id komt te staan.

Het onderdeel dat de URL en de methode vergelijkt met de routetabel en daarna de juiste code aanroept, heet een **router**. In onze API is dat de klasse `RequestHandler`. Die bekijken we in hoofdstuk 13.

### Tip

Het grote voordeel van een routetabel: je ziet in één bestand alle endpoints van je API. Wil je een endpoint toevoegen, dan voeg je één regel toe. De rest van de code hoeft niet te veranderen.

### Samenvatting

- Met **URL rewriting** maak je van `/threads/5` achter de schermen `index.php?resource=threads&id=5`.
- Alle requests komen binnen via één **front controller**: `index.php`.
- `.htaccess` is een instellingenbestand voor Apache; met **mod\_rewrite** herschrijf je URL's.
- **RewriteCond** is een voorwaarde voor de eerstvolgende **RewriteRule**. Flags: `[L]`, `[QSA]`, `[F]`.
- Een **routetabel** koppelt een methode en URL aan de code die moet worden uitgevoerd.

### Opdracht: Hoofdstuk 10

1. Wat is een front controller, en wat is het voordeel ervan?
2. Wat betekenen de flags `[L]`, `[QSA]` en `[F]`?
3. Wat wordt er van deze URL's gemaakt door onze `.htaccess`? a. `/forum-api/replies` b. `/forum-api/topics/12/` c. `/forum-api/threads/3?pagina=2` d. `/forum-api/composer.json`
4. Maak in je map `api-test` een bestand `dieren.php` dat `{"dier_id": ...}` teruggeeft met het getal uit `$_GET['id']`. Schrijf een `.htaccess`, zodat `http://localhost/api-test/dieren/3` wordt herschreven naar `dieren.php?id=3`. Alleen getallen mogen worden herschreven.
5. Je hebt je project in de map `htdocs/mijn-api` gezet in plaats van `htdocs/forum-api`. Welke regel in de `.htaccess` moet je aanpassen, en hoe?

# 11. Composer, namespaces en PSR-4

## Het probleem: veel bestanden

Onze API bestaat uit veel PHP-bestanden: controllers, models, de database-klasse en nog meer. Elk bestand bevat één klasse. Als het ene bestand een klasse uit een ander bestand gebruikt, moet PHP dat andere bestand eerst inladen. Zonder hulpmiddelen doe je dat met `require`:

### Zonder autoloader

```
<?php
require 'app/Database/Database.php';
require 'app/Models/Model.php';
require 'app/Models/ThreadModel.php';
require 'app/Models/TopicModel.php';
require 'app/Http/ApiResponse.php';
require 'app/Http/Controllers/Controller.php';
require 'app/Http/Controllers/ThreadController.php';
// ... en zo verder
```

Dat is veel werk en foutgevoelig. Vergeet je er één, dan krijg je de foutmelding *Class not found*. Daarom gebruiken we **automatisch laden** (*autoloading*): PHP laadt een bestand pas als een klasse echt nodig is, en zoekt zelf uit welk bestand dat is. Om dat mogelijk te maken, heb je twee dingen nodig: **namespaces** en een afspraak over waar de bestanden staan (**PSR-4**).

## Namespaces

In een grote school zitten misschien drie leerlingen die *Sam* heten. Om ze uit elkaar te houden, zeg je *Sam uit 4A* of *Sam uit 5C*. In PHP kan hetzelfde probleem ontstaan: twee klassen met dezelfde naam, bijvoorbeeld uit twee verschillende libraries. Daarom zet je klassen in een **namespace**: een soort achternaam of map voor klassen.

Bovenaan elk bestand geef je met `namespace` aan in welke namespace de klasse zit:

### app/Models/ThreadModel.php

```
<?php
namespace App\Models;

class ThreadModel extends Model
{
    // ...
}
```

De volledige naam van deze klasse is nu `App\Models\ThreadModel`. Namespaces schrijf je met een **backslash** `\`.

Wil je in een ander bestand een klasse uit een andere namespace gebruiken, dan zet je bovenaan een `use`-regel. Daarna kun je de korte naam gebruiken:

app/Http/Controllers/ThreadController.php (deel)

```
<?php
namespace App\Http\Controllers;

use App\Http\ApiResponse;
use App\Models\ThreadModel;

class ThreadController extends Controller
{
    public function index(): void
    {
        ApiResponse::sendResponse(ThreadModel::all());
    }
}
```

Klassen die in PHP zelf zitten, zoals `PDO`, staan in de hoofd-namespace. In een bestand met een eigen namespace schrijf je dan `use PDO;` bovenaan, of `\PDO` in je code.

#### Tip

Met `::class` krijg je de volledige naam van een klasse als tekst. `ThreadController::class` is hetzelfde als `'App\Http\Controllers\ThreadController'`. Dat gebruiken we in de routetabel. Het voordeel: VS Code waarschuwt je als je een tikfout maakt in de klassenaam.

## PSR-4: een afspraak over mappen

**PSR** staat voor *PHP Standards Recommendation*: afspraken die de PHP-wereld samen heeft gemaakt, zodat code van verschillende programmeurs goed samenwerkt. **PSR-4** is de afspraak over autoloading. Die zegt:

1. Je koppelt het begin van een namespace aan een map. Bij ons: de namespace `App\` hoort bij de map `app/`.
2. De rest van de namespace zijn **submappen**.
3. De naam van de klasse is de **naam van het bestand**, met `.php` erachter.

| Volledige klassenaam                              | Bestand                                               |
|---------------------------------------------------|-------------------------------------------------------|
| <code>App\Database\Database</code>                | <code>app/Database/Database.php</code>                |
| <code>App\Models\ThreadModel</code>               | <code>app/Models/ThreadModel.php</code>               |
| <code>App\Http\ApiResponse</code>                 | <code>app/Http/ApiResponse.php</code>                 |
| <code>App\Http\Controllers\ReplyController</code> | <code>app/Http/Controllers/ReplyController.php</code> |

Zie je het? Uit de naam van de klasse kan de autoloader precies afleiden in welk bestand hij staat.

#### Let op

Hoofdletters zijn belangrijk! Op Windows maakt het niet uit of je bestand `threadmodel.php` of `ThreadModel.php` heet. Maar de meeste echte servers draaien op Linux, en daar is het wél een verschil. Zorg er dus altijd voor dat de bestandsnaam precies gelijk is aan de naam van de klasse, en de mapnamen aan de namespace.

## Composer

**Composer** is het programma dat de autoloader voor je maakt. Daarnaast is Composer een **pakketbeheerder** voor PHP: je kunt er code van anderen (pakketten of *packages*) mee downloaden en gebruiken. Vergelijk het met npm voor JavaScript. De pakketten vind je op **packagist.org**.

### Composer installeren

1. Ga naar **getcomposer.org** en klik op **Download**. Op Windows download je **Composer-Setup.exe**.
2. Start de installatie. De installer vraagt waar PHP staat. Kies **C:\xampp\php\php.exe**.
3. Klik verder door de installatie heen.
4. Open een **nieuwe** terminal (of een nieuw terminalvenster in VS Code) en typ:

```
Terminal
composer --version
```

Zie je een versienummer? Dan werkt Composer.

#### Goed om te weten

Op een Mac kun je Composer installeren met de instructies op [getcomposer.org](https://getcomposer.org) onder *Command-line installation*. Gebruik je XAMPP op een Mac, dan moet de PHP van XAMPP bereikbaar zijn in je terminal. Vraag je docent om hulp als dat niet lukt.

## composer.json

De instellingen van Composer staan in het bestand **composer.json** in de hoofdmap van je project. Zo ziet het bestand van onze API eruit:

```

composer.json

{
    "name": "johan/php_api",
    "description": "Forum API geschreven in vanilla PHP",
    "type": "project",
    "authors": [
        {
            "name": "Johan Strootman",
            "email": "jj.strootman@alfa-college.nl"
        }
    ],
    "require": {
        "php": ">=8.1",
        "ext-pdo": "*",
        "ext-json": "*"
    },
    "autoload": {
        "psr-4": {
            "App\\": "app/"
        }
    }
}

```

- **name**, **description**, **type** en **authors** beschrijven het project.
- Onder **require** staat wat je project nodig heeft. Hier: PHP 8.1 of hoger, en de PHP-extensies **pdo** (voor de database) en **json**. Composer controleert of die aanwezig zijn.
- Onder **autoload** staat de PSR-4-afspraken: de namespace **App\\** hoort bij de map **app/**. De dubbele backslash **\\** is nodig omdat een backslash in JSON een speciaal teken is.

## De autoloader maken

Als **composer.json** klaar is, open je de terminal in de map van je project en typ je:

```

Terminal

composer dump-autoload

```

Composer maakt nu een map **vendor** met daarin het bestand **vendor/autoload.php**. Dat is de autoloader. Je hoeft hem alleen maar één keer in te laden, bovenaan **index.php**:

```

index.php (eerste regel)

require __DIR__ . '/vendor/autoload.php';

```

Vanaf nu wordt elke klasse automatisch ingeladen zodra je hem gebruikt. Geen lange lijst met **require** meer!

**\_\_DIR\_\_** is de map waar het huidige bestand in staat. Door **\_\_DIR\_\_** . ervoor te zetten, werkt het pad altijd, vanuit welke map het script ook gestart wordt.

 **Tip**

Maak je later een **nieuwe klasse** in de map `app`, dan hoef je `composer dump-autoload` niet opnieuw te doen. De PSR-4-autoloader zoekt het bestand gewoon op. Alleen als je de `autoload`-instellingen in `composer.json` verandert, moet je het opnieuw doen.

## Pakketten van anderen

Wil je een pakket van iemand anders gebruiken, dan typ je `composer require` met de naam van het pakket. Bijvoorbeeld een pakket om instellingen uit een `.env`-bestand te lezen:

```
Terminal
composer require vlucas/phpdotenv
```

Composer downloadt het pakket in `vendor`, zet het in `composer.json` onder `require` en maakt een bestand `composer.lock` met de precieze versies. Onze forum-API gebruikt geen pakketten van anderen; we gebruiken Composer alleen voor de autoloader.

### ⚠ Let op

Verander nooit zelf iets in de map `vendor`. Composer maakt die map en overschrijft hem als je `composer dump-autoload`, `composer install` of `composer update` doet. Deel je je project (bijvoorbeeld via GitHub), laat `vendor` dan weg. De ander typt gewoon `composer install` en krijgt alles terug.

### 📌 Samenvatting

- Met **autoloading** laadt PHP een klasse automatisch in wanneer je hem gebruikt.
- Een **namespace** is een soort map of achternaam voor klassen. Met `use` gebruik je een klasse uit een andere namespace.
- **PSR-4**: namespace-begin = map, rest van de namespace = submappen, klassenaam = bestandsnaam.
- **Composer** maakt de autoloader (`composer dump-autoload`) en beheert pakketten (`composer require`).
- In `index.php` laad je alleen `vendor/autoload.php` in.

### 📝 Opdracht: Hoofdstuk 11

1. Waarom gebruiken we namespaces?
2. In welk bestand moet volgens onze PSR-4-instelling de klasse `App\Http\Controllers\UserController` staan? En de klasse `App\Services\Mail\Mailer`?
3. Welke **namespace**-regel hoort bovenaan het bestand `app/Models/UserModel.php`?
4. Installeer Composer en controleer de versie met `composer --version`.
5. Een klasgenoot krijgt de fout `Class "App\Models\threadModel" not found`, terwijl het bestand `app/Models/ThreadModel.php` bestaat. Wat is er waarschijnlijk mis?
6. Waarom zet je de map `vendor` niet op GitHub?

## 12. CORS: wie mag je API gebruiken?

### De same-origin policy

Stel: je bent ingelogd bij je bank, en in een ander tabblad open je een louche website. Wat als die website met JavaScript stiekem een request doet naar de website van je bank, en je saldo uitleest? Dat zou heel gevaarlijk zijn.

Daarom hebben browsers een belangrijke veiligheidsregel: de **same-origin policy**. JavaScript op een website mag de response van een request naar een **andere origin** niet zomaar lezen.

Een **origin** is de combinatie van drie dingen: het **scheme** (`http` of `https`), de **host** en de **poort**. Zijn ze alle drie gelijk, dan is het dezelfde origin.

| Website                             | API                                     | Zelfde origin?                 |
|-------------------------------------|-----------------------------------------|--------------------------------|
| <code>http://localhost/forum</code> | <code>http://localhost/forum-api</code> | Ja: alleen het pad is anders   |
| <code>http://localhost:5173</code>  | <code>http://localhost/forum-api</code> | Nee: andere poort (5173 en 80) |
| <code>http://127.0.0.1:5500</code>  | <code>http://localhost/forum-api</code> | Nee: andere host               |
| <code>https://mijnforum.nl</code>   | <code>https://api.mijnforum.nl</code>   | Nee: andere host               |
| <code>http://mijnforum.nl</code>    | <code>https://mijnforum.nl</code>       | Nee: ander scheme              |

Als je een Vue- of React-app bouwt met Vite, draait die op `http://localhost:5173`. Je API draait op `http://localhost`. Dat zijn verschillende origins! Zonder extra instellingen blokkeert de browser dus de requests van je app naar je API.

### CORS: toestemming geven

Gelukkig kan de **server** toestemming geven. Dat heet **CORS**: *Cross-Origin Resource Sharing* (bronnen delen tussen verschillende origins). De server zet daarvoor speciale headers in de response. De browser kijkt naar die headers en beslist of JavaScript de response mag lezen.

De belangrijkste header is:

| Response-header                             |
|---------------------------------------------|
| <code>Access-Control-Allow-Origin: *</code> |

Het sterretje betekent: *elke website mag deze API gebruiken*. Je kunt ook één origin toestaan, bijvoorbeeld `Access-Control-Allow-Origin: http://localhost:5173`.

#### ⚠ Let op

CORS is een regel van de **browser**. Postman, een app op je telefoon of een ander PHP-script trekken zich er niets van aan. Werkt je request in Postman wel, maar in je website niet? Dan is CORS vaak de oorzaak. En omdat alleen de browser het controleert, is CORS geen beveiliging van je API: het beschermt de gebruikers van de browser.

## Simpele requests en de preflight

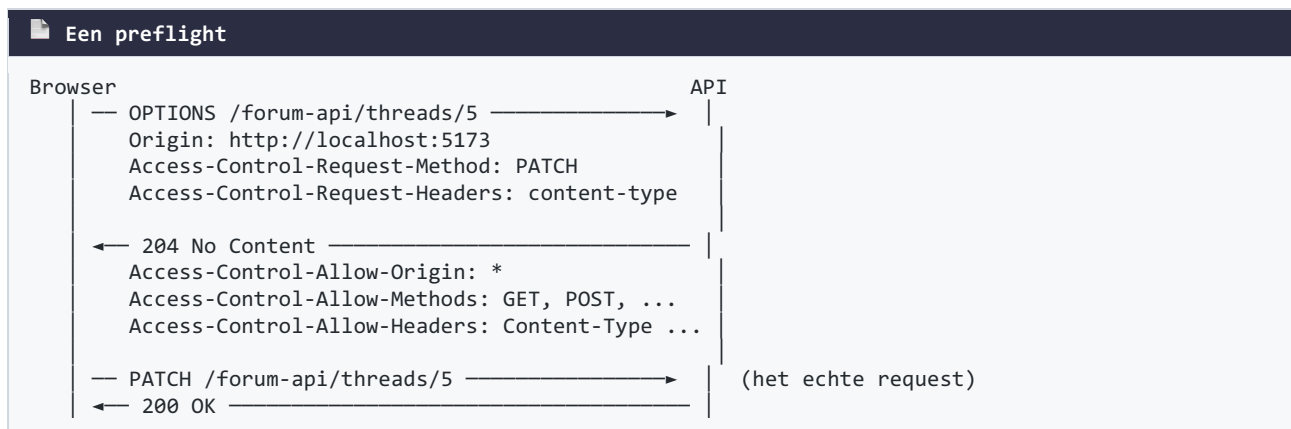
De browser maakt onderscheid tussen twee soorten requests.

Een **simpel request** is een GET of POST met alleen gewone headers en, bij POST, een gewoon formulierformaat. Die stuurt de browser meteen. Daarna kijkt hij of de response de header **Access-Control-Allow-Origin** heeft.

Alle andere requests zijn **niet simpel**, bijvoorbeeld:

- een request met de methode **PUT**, **PATCH** of **DELETE**;
- een request met **Content-Type: application/json**;
- een request met een header als **Authorization**.

Voor zo'n request vraagt de browser eerst toestemming, met een **preflight**: een **OPTIONS**-request naar dezelfde URL. Pas als het antwoord goed is, stuurt de browser het echte request.



De headers die de server bij een preflight terugstuurt:

| Header                              | Betekenis                                                                                                                   |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Access-Control-Allow-Origin</b>  | Welke origin(s) de API mogen gebruiken.                                                                                     |
| <b>Access-Control-Allow-Methods</b> | Welke methodes mogen.                                                                                                       |
| <b>Access-Control-Allow-Headers</b> | Welke request-headers de client mag meesturen.                                                                              |
| <b>Access-Control-Max-Age</b>       | Hoeveel seconden de browser dit antwoord mag onthouden, zodat hij niet voor elk request een nieuwe preflight hoeft te doen. |

Daarnaast is er nog **Access-Control-Expose-Headers**. Standaard mag JavaScript maar een paar response-headers lezen. Met deze header geef je toestemming voor meer, bijvoorbeeld voor **Location**.

## CORS in onze forum-API

In de klasse `ApiResponse` van onze API zitten twee functies voor CORS:

- `sendDefaultHeaders()` zet bij **elke** response `Access-Control-Allow-Origin: *` en `Access-Control-Expose-Headers: Location`.
- `sendCORSHeaders()` beantwoordt een **preflight** met `204 No Content` en alle `Access-Control-Allow-...`-headers.

In `RequestHandler` wordt een OPTIONS-request als allereerste afgehandeld, nog vóórdat er naar een route wordt gezocht:

Uit `app/Http/RequestHandler.php`

```
// 1. Een OPTIONS-request is een CORS-preflight: meteen beantwoorden
if ($this->requestMethod === self::OPTIONS) {
    ApiResponse::sendCORSHeaders();
}
```

### Goed om te weten

In de originele versie van het project gebeurde dat pas **na** het zoeken naar een route. Omdat er geen routes voor OPTIONS waren, kreeg elke preflight een **404**, en werkte PUT, PATCH en DELETE vanuit een browser nooit. Daarom staat de controle nu helemaal vooraan.

## Zelf proberen

Je kunt CORS zelf zien gebeuren. Installeer in VS Code de extensie **Live Server**. Maak ergens op je computer (niet in httdocs) een map `cors-test` met het bestand `index.html`:

```
cors-test/index.html

<!DOCTYPE html>
<html lang="nl">
<head>
  <meta charset="UTF-8">
  <title>CORS-test</title>
</head>
<body>
  <button id="ophalen">GET hallo.php</button>
  <button id="versturen">POST JSON naar echo.php</button>
  <pre id="uitvoer"></pre>

  <script>
    const uitvoer = document.getElementById('uitvoer')

    document.getElementById('ophalen').addEventListener('click', async () => {
      try {
        const response = await fetch('http://localhost/api-test/hallo.php')
        uitvoer.textContent = JSON.stringify(await response.json(), null, 2)
      } catch (fout) {
        uitvoer.textContent = 'Mislukt! Kijk in de console (F12).'
      }
    })

    document.getElementById('versturen').addEventListener('click', async () => {
      try {
        const response = await fetch('http://localhost/api-test/echo.php', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ naam: 'Sam', klas: '4B' })
        })
        uitvoer.textContent = JSON.stringify(await response.json(), null, 2)
      } catch (fout) {
        uitvoer.textContent = 'Mislukt! Kijk in de console (F12).'
      }
    })
  </script>
</body>
</html>
```

Klik met de rechtermuisknop op `index.html` en kies **Open with Live Server**. De pagina opent op `http://127.0.0.1:5500`: een andere origin dan je API. Klik op de knoppen en open de console (F12). Je ziet een rode melding zoals:

```
Console

Access to fetch at 'http://localhost/api-test/hallo.php' from origin
'http://127.0.0.1:5500' has been blocked by CORS policy: No
'Access-Control-Allow-Origin' header is present on the requested resource.
```

De browser heeft de response gewoon gekregen, maar geeft hem niet aan JavaScript, omdat de API geen toestemming gaf. In de opdrachten los je dit op.

### Samenvatting

- De **same-origin policy** zorgt dat JavaScript geen responses van een andere **origin** mag lezen.
- Een origin = scheme + host + poort.
- Met **CORS** geeft de server toestemming via headers, zoals **Access-Control-Allow-Origin**.
- Bij niet-simpele requests (PUT, PATCH, DELETE, JSON) stuurt de browser eerst een **preflight**: een OPTIONS-request.
- CORS wordt alleen door de browser gecontroleerd, niet door Postman.

### Opdracht: Hoofdstuk 12

1. Wat is een origin? Hebben <http://localhost:8080> en <http://localhost:3000> dezelfde origin?
2. Waarom werkt een request naar je API wel in Postman, maar niet vanuit JavaScript op een andere website?
3. Welke van deze requests veroorzaken een preflight? a. **GET /threads** b. **DELETE /threads/5** c. **POST /threads** met **Content-Type: application/json** d. **POST /threads** met gewone formulierdata
4. Maak de CORS-test na en bekijk de foutmelding in de console. Voeg daarna in **hallo.php** de juiste header toe, zodat de knop **GET hallo.php** werkt.
5. Pas **echo.php** aan, zodat ook de knop **POST JSON naar echo.php** werkt. Denk aan de preflight!
6. Waarom zet onze API de header **Access-Control-Expose-Headers: Location**?

## 13. Eindproject: de forum-API

### Wat gaan we bouwen?

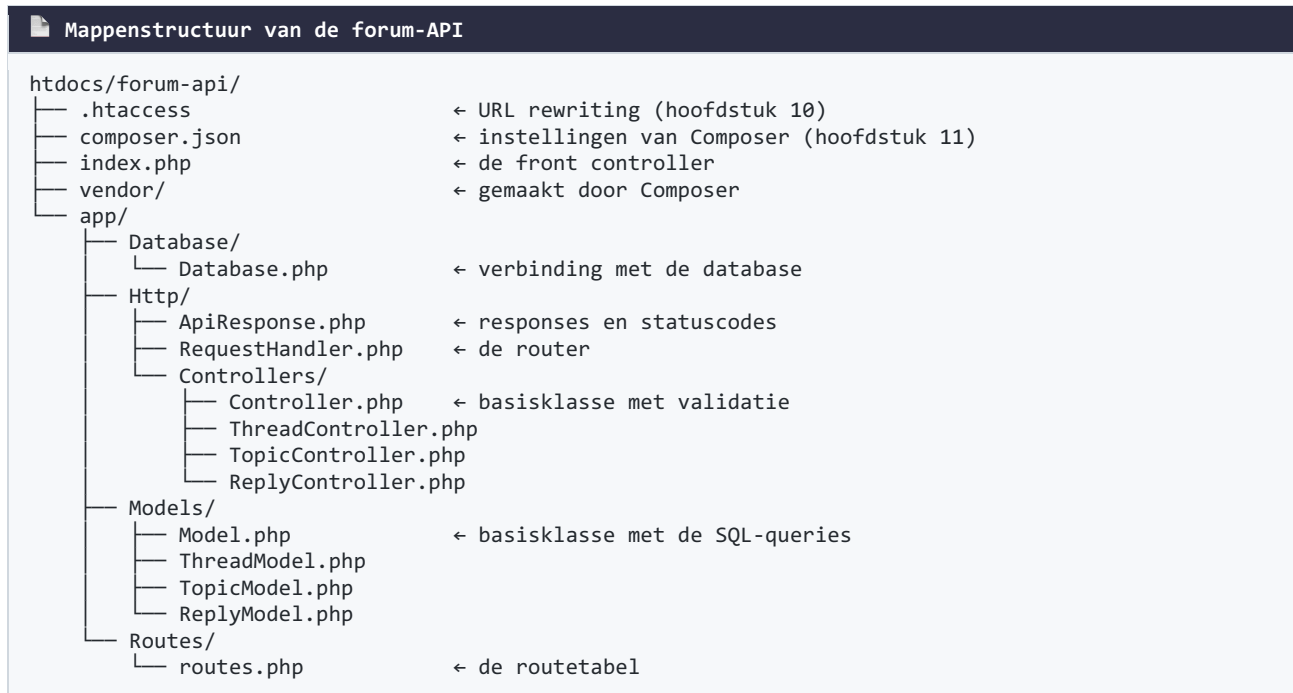
Nu komt alles samen! In dit hoofdstuk bouw je stap voor stap de complete forum-API. Aan het einde heeft je API deze endpoints:

| Methode | URL                       | Actie                                   | Succes |
|---------|---------------------------|-----------------------------------------|--------|
| GET     | /threads                  | Alle threads ophalen                    | 200    |
| GET     | /threads/{id}             | Eén thread ophalen                      | 200    |
| POST    | /threads                  | Een thread aanmaken                     | 201    |
| PUT     | /threads/{id}             | Een thread helemaal vervangen           | 200    |
| PATCH   | /threads/{id}             | Een thread gedeeltelijk wijzigen        | 200    |
| DELETE  | /threads/{id}             | Een thread verwijderen                  | 204    |
|         | /topics en /topics/{id}   | Dezelfde zes endpoints voor topics      |        |
|         | /replies en /replies/{id} | Dezelfde zes endpoints voor replies     |        |
| GET     | /                         | Alle threads (de 'homepage' van de API) | 200    |
| OPTIONS | elke URL                  | CORS-preflight                          | 204    |

Bij fouten geeft de API de juiste foutcode: 400, 404, 405, 422 of 500.

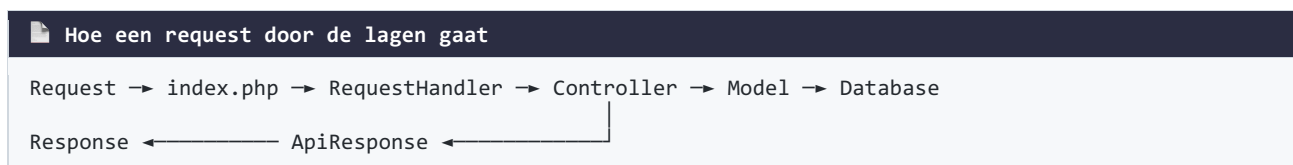
## De mappenstructuur

Zo ziet het project eruit als het klaar is:



De code is verdeeld in lagen. Elke laag heeft één taak:

- De **router** (**RequestHandler**) kijkt welke code bij het request hoort.
- Een **controller** bepaalt wat er moet gebeuren: gegevens controleren, het model aanroepen en de juiste response sturen.
- Een **model** praat met de database.
- **ApiResponse** maakt de JSON-responses.



## Wat is er veranderd aan het originele project?

Dit project is gebaseerd op een bestaand voorbeeldproject. Dat project was nog niet af, en er zaten een paar fouten in. In dit boek is het afgemaakt en verbeterd. Hieronder zie je wat er veranderd is en waarom. Het is leerzaam om te zien welke fouten er in echte code kunnen zitten!

| In het originele project                                                                                                                 | Wat is er veranderd                                                                       | Waarom                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| PUT, PATCH en DELETE deden niets (lege methodes).                                                                                        | Alle methodes zijn gebouwd in controllers en models.                                      | Een API moet alle CRUD-acties kunnen.                                                                     |
| <code>handleDeleteRequest(int \$id)</code> werd zonder id aangeroepen.                                                                   | Eén algemene manier om controllers aan te roepen, met de juiste argumenten.               | Dat gaf een fatale fout. De vijf bijna gelijke <code>handle...Request</code> -methodes zijn samengevoegd. |
| Een OPTIONS-request (preflight) kreeg een <code>404</code> .                                                                             | OPTIONS wordt als eerste afgehandeld.                                                     | Anders werkt CORS niet voor PUT, PATCH en DELETE vanuit een browser.                                      |
| URL's in het enkelvoud ( <code>/thread/1</code> ) en meervoud ( <code>/threads</code> ) door elkaar.                                     | Overall meervoud: <code>/threads</code> en <code>/threads/1</code> .                      | Zo is het RESTful (hoofdstuk 8).                                                                          |
| De DELETE-routes voor topics en replies wezen naar <code>update</code> .                                                                 | Ze wijzen naar <code>destroy</code> .                                                     | Anders werd er bij DELETE iets gewijzigd in plaats van verwijderd.                                        |
| <code>ThreadModel::create()</code> gebruikte de kolom <code>description</code> .                                                         | De kolom heet <code>content</code> , zoals in de database.                                | De query mislukte, maar niemand zag het.                                                                  |
| <code>Database</code> ving fouten op en deed er niets mee.                                                                               | PDO gooit een exception; <code>index.php</code> vangt die op en stuurt <code>500</code> . | Stille fouten zijn het moeilijkst om te vinden.                                                           |
| Databasenaam <code>forum-sd</code> en wachtwoord <code>root</code> .                                                                     | <code>forum_api_vb</code> en een leeg wachtwoord.                                         | Zo is het ingesteld in XAMPP en de SQL-dump.                                                              |
| Eigen statuscodes <code>210</code> en <code>211</code> .                                                                                 | Officiële codes <code>200</code> , <code>201</code> en <code>204</code> .                 | Verzin nooit eigen statuscodes (hoofdstuk 7).                                                             |
| <code>header("HTTP/1.1 \$code \$msg")</code> .                                                                                           | <code>http_response_code(\$code)</code> .                                                 | Werkt altijd goed, ook met andere HTTP-versies.                                                           |
| <code>count</code> in <code>meta</code> bij één item telde de velden, en een onbekend id gaf <code>[false]</code> met <code>200</code> . | <code>count</code> alleen bij lijsten; een onbekend id geeft <code>404</code> .           | De response moet kloppen.                                                                                 |
| Alleen <code>\$_POST</code> werd gelezen.                                                                                                | JSON uit de body wordt gelezen, en formulierdata bij PUT en PATCH.                        | API's werken met JSON, en PHP vult <code>\$_POST</code> niet bij PUT en PATCH.                            |
| Geen controle van de invoer.                                                                                                             | Validatie met <code>422</code> en per veld een melding.                                   | Nooit zomaar vertrouwen wat een client stuurt.                                                            |
| <code>@include</code> in <code>index.php</code> .                                                                                        | <code>require</code> , zonder <code>@</code> .                                            | Het <code>@</code> verstopt foutmeldingen.                                                                |
| Elk model had dezelfde code.                                                                                                             | Een basisklasse <code>Model</code> , waar elk model van erft.                             | Minder dubbele code.                                                                                      |

| In het originele project                                                                                                | Wat is er veranderd                                                                                                                                                  | Waarom                                           |
|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| In <code>.htaccess</code> gold de <code>RewriteCond</code> alleen voor de eerste regel, en <code>RewriteBase /</code> . | De voorwaarden staan bij beide regels; <code>RewriteBase /forum-api/</code> ; <code>app</code> , <code>vendor</code> en <code>composer.json</code> zijn afgeschermd. | Zie hoofdstuk 10.                                |
| Lege klassen <code>LoginController</code> en <code>RegisterController</code> .                                          | Weggelaten.                                                                                                                                                          | Inloggen valt buiten dit boek; zie hoofdstuk 15. |

## Stap 1: het project aanmaken

Maak in `htdocs` de map `forum-api` en open die in VS Code. Maak de mappen `app`, `app/Database`, `app/Http`, `app/Http/Controllers`, `app/Models` en `app/Routes`.

Maak daarna `composer.json` aan zoals in hoofdstuk 11, en de `.htaccess` zoals in hoofdstuk 10. Open de terminal in VS Code en maak de autoloader:

```

Terminal
composer dump-autoload

```

## Stap 2: index.php, de voordeur

Maak `index.php` in de hoofdmap:

```

index.php

<?php
require __DIR__ . '/vendor/autoload.php';

use App\Http\ApiResponse;
use App\Http\RequestHandler;

try {
    $routes = require __DIR__ . '/app/Routes/routes.php';

    $requestHandler = new RequestHandler($routes);
    $requestHandler->handleRequest();
} catch (Throwable $e) {
    // De echte fout schrijven we in het logbestand van de server,
    // de client krijgt alleen een algemene melding.
    error_log($e->getMessage());

    ApiResponse::sendError(
        ApiResponse::HTTP_STATUS_SERVER_ERROR,
        'Er ging iets mis op de server. Probeer het later opnieuw.'
    );
}

```

Dit bestand doet drie dingen:

1. Het laadt de **autoloader** van Composer.
2. Het laadt de **router** (het bestand geeft met `return` een array terug) en geeft die aan de `RequestHandler`.
3. Alles staat in een **try/catch**. Gaat er ergens in de API iets onverwachts mis, bijvoorbeeld omdat de database uitstaat, dan springt PHP naar het `catch`-blok. De echte fout wordt met `error_log()` in het logbestand van Apache geschreven (in XAMPP: `C:\xampp\apache\logs\error.log`), en de client krijgt een nette **500**-response in JSON.

`Throwable` vangt alle soorten fouten op: zowel exceptions als ernstige PHP-fouten, zoals het aanroepen van een methode die niet bestaat.

## Stap 3: de database-klasse

Maak `app/Database/Database.php`:

`app/Database/Database.php`

```
<?php
namespace App\Database;

use PDO;
use PDOStatement;

class Database
{
    // Instellingen voor XAMPP: gebruiker root zonder wachtwoord
    private string $dbHost = '127.0.0.1';
    private string $dbName = 'forum_api_vb';
    private string $dbUser = 'root';
    private string $dbPass = '';

    private PDO $dbConnection;
    private ?PDOStatement $dbStatement = null;

    public function __construct()
    {
        $this->dbConnection = new PDO(
            "mysql:host={$this->dbHost};dbname={$this->dbName};charset=utf8mb4",
            $this->dbUser,
            $this->dbPass,
            [
                // Bij een fout gooit PDO een exception, in plaats van stil te blijven
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                // Rijen komen terug als array met kolomnamen: ['id' => 1, 'title' => '...']
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                // Echte prepared statements; getallen komen terug als getal
                PDO::ATTR_EMULATE_PREPARES => false,
            ]
        );
    }

    public function query(string $sql, array $args = []): void
    {
        $this->dbStatement = $this->dbConnection->prepare($sql);
        $this->dbStatement->execute($args);
    }

    public function insert(string $sql, array $args = []): int
    {
        $this->query($sql, $args);

        return (int) $this->dbConnection->lastInsertId();
    }

    // Eén rij ophalen, of null als er niets gevonden is
    public function get(): ?array
    {
        $row = $this->dbStatement?->fetch();

        if ($row === false || $row === null) {
            return null;
        }

        return $row;
    }

    // Alle rijen ophalen
    public function getAll(): array
```

```

{
    return $this->dbStatement?->fetchAll() ?? [];
}

// Hoeveel rijen zijn er veranderd door de laatste query?
public function rowCount(): int
{
    return $this->dbStatement?->rowCount() ?? 0;
}
}

```

We gebruiken **PDO** om met de database te praten. PDO is ingebouwd in PHP. Bij het maken van de verbinding geven we drie opties mee:

- **ERRMODE\_EXCEPTION**: als er iets misgaat, gooit PDO een exception. In het originele project werden fouten stil opgevangen, waardoor je nooit zag dát er iets misging.
- **FETCH\_ASSOC**: rijen komen terug als array met de kolomnamen als sleutel. Die kunnen we zo in JSON omzetten.
- **EMULATE\_PREPARES => false**: echte prepared statements. Een fijn bijeffect: getallen uit de database komen terug als echte getallen (`"user_id": 11`) in plaats van als tekst (`"user_id": "11"`).

De methode `query()` gebruikt **prepared statements**: de SQL en de waarden worden apart naar de database gestuurd. In de SQL staat een plekje zoals `:id`, en de echte waarde gaat mee in `$args`.

#### ⚠ Let op

Zet waarden van de client **nooit** direct in je SQL, zoals `"SELECT * FROM threads WHERE id = " . $_GET['id']`. Een kwaadwillende client kan dan zelf SQL-code meesturen en bijvoorbeeld je hele database leeghalen. Dat heet **SQL-injectie**. Met prepared statements kan dat niet, omdat de waarden nooit als SQL worden gelezen.

De `?->` heet de **nullsafe operator**. `$this->dbStatement?->fetch()` betekent: roep `fetch()` alleen aan als `$this->dbStatement` niet `null` is; anders is de uitkomst `null`.

## Stap 4: de models

Alle drie de models (threads, topics en replies) doen hetzelfde: alles ophalen, één ophalen, aanmaken, wijzigen en verwijderen. Alleen de tabel en de velden verschillen. Daarom maken we één **basisklasse** `Model` met alle code, en drie kleine klassen die daarvan **erven**. Maak `app/Models/Model.php`:

`app/Models/Model.php`

```
<?php
namespace App\Models;

use App\Database\Database;

abstract class Model
{
    // Elke model-klasse vult deze twee properties zelf in
    protected static string $table;
    protected static array $fillable;

    private static ?Database $db = null;

    // Geeft de databaseverbinding; maakt hem aan als dat nog niet gebeurd is
    protected static function db(): Database
    {
        if (self::$db === null) {
            self::$db = new Database();
        }

        return self::$db;
    }

    // Houdt alleen de velden over die in $fillable staan
    protected static function onlyFillable(array $data): array
    {
        return array_intersect_key($data, array_flip(static::$fillable));
    }

    public static function all(): array
    {
        $table = static::$table;

        self::db()->query("SELECT * FROM `{$table}`");

        return self::db()->getAll();
    }

    public static function find(int $id): ?array
    {
        $table = static::$table;

        self::db()->query("SELECT * FROM `{$table}` WHERE `id` = :id", [ 'id' => $id ]);

        return self::db()->get();
    }

    public static function create(array $data): array
    {
        $table = static::$table;

        $data = static::onlyFillable($data);
        $data['created_at'] = date('Y-m-d H:i:s');
        $data['updated_at'] = date('Y-m-d H:i:s');

        // Bijv. ['title', 'content'] wordt "`title`, `content`" en ":title, :content"
```

```

        $columns = array_keys($data);
        $columnList = '`' . implode('`, `', $columns) . '`';
        $placeholders = ':' . implode(':', $columns);

        $id = self::db()->insert(
            "INSERT INTO `{$table}` ({$columnList}) VALUES ({$placeholders})",
            $data
        );

        return static::find($id);
    }

    public static function update(int $id, array $data): ?array
    {
        $table = static::$table;

        $data = static::onlyFillable($data);
        $data['updated_at'] = date('Y-m-d H:i:s');

        // Bijv. "`title` = :title, `updated_at` = :updated_at"
        $sets = [];
        foreach (array_keys($data) as $column) {
            $sets[] = "`{$column}` = :{$column}";
        }

        $data['id'] = $id;

        self::db()->query(
            "UPDATE `{$table}` SET " . implode(' ', $sets) . " WHERE `id` = :id",
            $data
        );

        return static::find($id);
    }

    // Geeft true als er echt een rij is verwijderd
    public static function delete(int $id): bool
    {
        $table = static::$table;

        self::db()->query("DELETE FROM `{$table}` WHERE `id` = :id", [ 'id' => $id ]);

        return self::db()->rowCount() > 0;
    }
}

```

Een paar dingen die misschien nieuw zijn:

- **abstract class** betekent dat je van **Model** zelf geen object kunt maken. Het is alleen bedoeld als basis voor andere klassen.
- Alle methodes zijn **static**: je roept ze aan op de klasse, zonder eerst een object te maken. Bijvoorbeeld **ThreadModel::find(5)**.
- **self::** verwijst altijd naar de klasse **Model** zelf. **static::** verwijst naar de klasse waarop je de methode **aanroept**. Roep je **ThreadModel::all()** aan, dan is **static::\$table** gelijk aan **'threads'**. Roep je **ReplyModel::all()** aan, dan is het **'replies'**. Zo werkt dezelfde code voor elke tabel.
- De databaseverbinding wordt pas gemaakt als hij voor het eerst nodig is, en daarna hergebruikt.

In `create()` en `update()` wordt de SQL zelf opgebouwd uit de namen van de velden. Stuur de client `title` en `content`, dan wordt de query `INSERT INTO threads (title, content, created_at, updated_at) VALUES (:title, :content, :created_at, :updated_at)`.

### ⚠ Let op

Omdat de kolomnamen in de SQL komen, mogen die nooit zomaar van de client komen. Daarom houdt `onlyFillable()` alleen de velden over die in de lijst `$fillable` staan. Stuur een client ook `"id": 1` of `"created_at": "2000-01-01"` mee, dan worden die genegeerd. Dit beschermt tegen **mass assignment**: een client die velden aanpast die hij niet mag aanpassen.

Nu de drie models. Ze zeggen alleen welke tabel ze gebruiken en welke velden gevuld mogen worden:

#### app/Models/ThreadModel.php

```
<?php
namespace App\Models;

class ThreadModel extends Model
{
    protected static string $table = 'threads';
    protected static array $fillable = [ 'title', 'content', 'user_id' ];
}
```

#### app/Models/TopicModel.php

```
<?php
namespace App\Models;

class TopicModel extends Model
{
    protected static string $table = 'topics';
    protected static array $fillable = [ 'title', 'content', 'thread_id', 'user_id' ];
}
```

#### app/Models/ReplyModel.php

```
<?php
namespace App\Models;

class ReplyModel extends Model
{
    protected static string $table = 'replies';
    protected static array $fillable = [ 'content', 'topic_id', 'user_id' ];
}
```

Met `extends Model` erft elke klasse alle methodes van `Model`. `ThreadModel::find(5)` werkt dus, ook al staat `find()` niet in `ThreadModel.php`.

## Stap 5: ApiResponse, de responses

Maak `app/Http/ApiResponse.php`:

`app/Http/ApiResponse.php`

```
<?php
namespace App\Http;

class ApiResponse
{
    // Algemene informatie over de API
    public const API_VERSION = '1.0';
    public const API_NAME = 'forum_api';

    // SUCCESS CODES (2xx)
    public const HTTP_STATUS_OK = 200;
    public const HTTP_STATUS_CREATED = 201;
    public const HTTP_STATUS_NO_CONTENT = 204;

    // CLIENT ERROR CODES (4xx)
    public const HTTP_STATUS_BAD_REQUEST = 400;
    public const HTTP_STATUS_UNAUTHORIZED = 401;
    public const HTTP_STATUS_FORBIDDEN = 403;
    public const HTTP_STATUS_NOT_FOUND = 404;
    public const HTTP_STATUS_METHOD_NOT_ALLOWED = 405;
    public const HTTP_STATUS_UNPROCESSABLE = 422;

    // SERVER ERROR CODES (5xx)
    public const HTTP_STATUS_SERVER_ERROR = 500;
    public const HTTP_STATUS_NOT_IMPLEMENTED = 501;
    public const HTTP_STATUS_SERVICE_NOT_AVAIL = 503;

    // De officiële tekst bij elke statuscode
    private const STATUS_MESSAGES = [
        200 => 'OK',
        201 => 'Created',
        204 => 'No Content',
        400 => 'Bad Request',
        401 => 'Unauthorized',
        403 => 'Forbidden',
        404 => 'Not Found',
        405 => 'Method Not Allowed',
        422 => 'Unprocessable Content',
        500 => 'Internal Server Error',
        501 => 'Not Implemented',
        503 => 'Service Unavailable',
    ];

    // Antwoord op een CORS-preflight (OPTIONS-request)
    public static function sendCORSHeaders(): void
    {
        header('Access-Control-Allow-Origin: *');
        header('Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS');
        header('Access-Control-Allow-Headers: Content-Type, Accept, Authorization');
        header('Access-Control-Max-Age: 86400');

        self::sendStatusCode(self::HTTP_STATUS_NO_CONTENT);
        exit;
    }

    // Headers die bij elke gewone response horen
    public static function sendDefaultHeaders(): void
    {
        header('Access-Control-Allow-Origin: *');
        header('Access-Control-Expose-Headers: Location');
        header('Content-Type: application/json; charset=utf-8');
    }
}
```

```

public static function sendStatusCode(int $code): void
{
    http_response_code($code);
}

public static function getStatusMessage(int $code): string
{
    return self::STATUS_MESSAGES[$code] ?? '';
}

// Maakt een volledige URL naar een resource, bijv. voor de Location-header
public static function url(string $path): string
{
    $scheme = (!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off') ? 'https' : 'http';
    $basePath = rtrim(dirname($_SERVER['SCRIPT_NAME']), '/\\');

    return "{$scheme}://{$_SERVER['HTTP_HOST']}{$basePath}/{ $path}";
}

// Het meta-deel dat in elke response zit
private static function buildMeta(int $code, mixed $data = null): array
{
    $meta = [
        'api_version' => self::API_VERSION,
        'api_name' => self::API_NAME,
        'status' => $code,
        'status_message' => self::getStatusMessage($code),
    ];

    // Alleen bij een lijst (een array met sleutels 0, 1, 2, ...) tellen we de items
    if (is_array($data) && array_is_list($data)) {
        $meta['count'] = count($data);
    }

    return $meta;
}

private static function toJson(array $response): string
{
    return json_encode(
        $response,
        JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES
    );
}

// Stuurt een succesvolle response en stopt het script
public static function sendResponse(mixed $data, int $code = self::HTTP_STATUS_OK): void
{
    self::sendDefaultHeaders();
    self::sendStatusCode($code);

    // Bij 204 No Content mag er geen body worden meegestuurd
    if ($code !== self::HTTP_STATUS_NO_CONTENT) {
        echo self::toJson([
            'meta' => self::buildMeta($code, $data),
            'data' => $data,
        ]);
    }

    exit;
}

// Stuurt een foutmelding en stopt het script
public static function sendError(int $code, string $message, array $details = []): void
{
    self::sendDefaultHeaders();

```

```

        self::sendStatusCode($code);

        $error = [ 'message' => $message ];

        if (!empty($details)) {
            $error['details'] = $details;
        }

        echo self::toJson([
            'meta' => self::buildMeta($code),
            'error' => $error,
        ]);

        exit;
    }
}

```

Deze klasse zorgt dat alle responses dezelfde structuur hebben (hoofdstuk 9):

- Bovenaan staan **constanten** voor alle statuscodes. In je code schrijf je `ApiResponse::HTTP_STATUS_NOT_FOUND` in plaats van het losse getal `404`. Dat leest makkelijker en voorkomt tikfouten.
- `STATUS_MESSAGES` bevat de officiële tekst bij elke code. Die komt in `status_message` in `meta`.
- `sendCORSHeaders()` beantwoordt de preflight en `sendDefaultHeaders()` stuurt de headers die bij elke response horen (hoofdstuk 12).
- `buildMeta()` maakt het `meta`-deel. `array_is_list()` controleert of de data een lijst is; alleen dan komt `count` erbij.
- `sendResponse()` stuurt een succesvolle response met `meta` en `data`. Bij `204` wordt er geen body gestuurd.
- `sendError()` stuurt een foutmelding met `meta` en `error`, eventueel met `details`.
- `url()` maakt een volledige URL, zoals `http://localhost/forum-api/threads/23`, voor de `Location`-header.

Beide `send`-methodes eindigen met `exit`. Zodra er een response is verstuurd, stopt het script. Zo kan er nooit per ongeluk een tweede response achteraan komen.

## Stap 6: de controllers

Eerst de basisklasse **Controller**. Die bevat de methode **validate()**, die alle controllers gebruiken om de invoer van de client te controleren. Maak **app/Http/Controllers/Controller.php**:

 **app/Http/Controllers/Controller.php**

```
<?php
namespace App\Http\Controllers;

abstract class Controller
{
    // Controleert de data met regels zoals 'required|string|max:100'.
    // $partial = true bij PATCH: dan controleren we alleen de meegestuurde velden.
    // Geeft een array met foutmeldingen terug (leeg = alles in orde).
    protected function validate(array $data, array $rules, bool $partial = false): array
    {
        $errors = [];

        // Bij PATCH moet er minstens één bekend veld zijn meegestuurd
        if ($partial && count(array_intersect_key($data, $rules)) === 0) {
            $errors['body'] = 'Stuur minstens één van deze velden mee: ' . implode(', ',
array_keys($rules)) . '.';
        }

        return $errors;
    }

    foreach ($rules as $field => $ruleString) {
        $fieldRules = explode('|', $ruleString);
        $isFilled = isset($data[$field]) && $data[$field] !== '';

        // Veld ontbreekt of is leeg
        if (!$isFilled) {
            $isMissingInPatch = $partial && !array_key_exists($field, $data);

            if (in_array('required', $fieldRules) && !$isMissingInPatch) {
                $errors[$field] = "Het veld {$field} is verplicht.";
            }

            continue;
        }

        $value = $data[$field];

        foreach ($fieldRules as $rule) {
            if ($rule === 'string' && !is_string($value)) {
                $errors[$field] = "Het veld {$field} moet tekst zijn.";
                break;
            }

            if ($rule === 'int' && filter_var($value, FILTER_VALIDATE_INT, ['options' =>
['min_range' => 1]]) === false) {
                $errors[$field] = "Het veld {$field} moet een positief geheel getal zijn.";
                break;
            }

            if (str_starts_with($rule, 'max:')) {
                $max = (int) substr($rule, 4);

                if (is_string($value) && mb_strlen($value) > $max) {
                    $errors[$field] = "Het veld {$field} mag maximaal {$max} tekens bevatten.";
                    break;
                }
            }
        }
    }
}
```

```

    }
    return $errors;
}

```

Elke controller geeft voor elk veld **regels** mee, gescheiden door **|**:

| Regel                 | Betekenis                                                                                  |
|-----------------------|--------------------------------------------------------------------------------------------|
| <code>required</code> | Het veld moet aanwezig zijn en mag niet leeg zijn.                                         |
| <code>string</code>   | Het veld moet tekst zijn.                                                                  |
| <code>int</code>      | Het veld moet een positief geheel getal zijn (ook <code>"3"</code> uit een formulier mag). |
| <code>max:100</code>  | Het veld mag maximaal 100 tekens lang zijn.                                                |

Bij **PATCH** wordt `validate()` aangeroepen met `$partial = true`. Dan worden alleen de velden gecontroleerd die de client heeft meegestuurd, want bij PATCH hoeft je niet alles mee te sturen. Wel moet er minstens één bekend veld bij zitten.

`validate()` geeft een array met foutmeldingen terug. Is die leeg, dan is alles in orde.

Nu de `ThreadController`. Maak `app/Http/Controllers/ThreadController.php`:

`app/Http/Controllers/ThreadController.php`

```
<?php
namespace App\Http\Controllers;

use App\Http\ApiResponse;
use App\Models\ThreadModel;

class ThreadController extends Controller
{
    // Regels voor de velden van een thread
    private const RULES = [
        'title' => 'required|string|max:100',
        'content' => 'required|string',
        'user_id' => 'required|int',
    ];

    // GET /threads
    public function index(): void
    {
        ApiResponse::sendResponse(ThreadModel::all());
    }

    // GET /threads/{id}
    public function show(int $id): void
    {
        $thread = ThreadModel::find($id);

        if ($thread === null) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Thread {$id} bestaat niet.");
        }

        ApiResponse::sendResponse($thread);
    }

    // POST /threads
    public function create(array $request): void
    {
        $errors = $this->validate($request, self::RULES);

        if (!empty($errors)) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_UNPROCESSABLE, 'De invoer is niet geldig.', $errors);
        }

        $thread = ThreadModel::create($request);

        header('Location: ' . ApiResponse::url('threads/' . $thread['id']));
        ApiResponse::sendResponse($thread, ApiResponse::HTTP_STATUS_CREATED);
    }

    // PUT /threads/{id} : alle velden zijn verplicht
    public function update(int $id, array $request): void
    {
        if (ThreadModel::find($id) === null) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Thread {$id} bestaat niet.");
        }

        $errors = $this->validate($request, self::RULES);

        if (!empty($errors)) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_UNPROCESSABLE, 'De invoer is niet geldig.', $errors);
        }
    }
}
```

```

        ApiResponse::sendResponse(ThreadModel::update($id, $request));
    }

    // PATCH /threads/{id} : alleen de meegestuurde velden worden aangepast
    public function patch(int $id, array $request): void
    {
        if (ThreadModel::find($id) === null) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Thread {$id} bestaat niet.");
        }

        $errors = $this->validate($request, self::RULES, true);

        if (!empty($errors)) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_UNPROCESSABLE, 'De invoer is niet
geldig.', $errors);
        }

        ApiResponse::sendResponse(ThreadModel::update($id, $request));
    }

    // DELETE /threads/{id}
    public function destroy(int $id): void
    {
        if (!ThreadModel::delete($id)) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Thread {$id} bestaat niet.");
        }

        ApiResponse::sendResponse(null, ApiResponse::HTTP_STATUS_NO_CONTENT);
    }
}

```

Elke methode hoort bij één endpoint. Kijk goed welke statuscode er wordt gebruikt:

- `index()` haalt alle threads op en stuurt **200**.
- `show()` zoekt één thread. Bestaat die niet, dan **404**; anders **200**.
- `create()` controleert eerst de invoer. Klopt die niet, dan **422** met per veld een melding. Anders wordt de thread aangemaakt en krijgt de client **201 Created**, met de nieuwe thread in de body en de URL ervan in de **Location**-header.
- `update()` (PUT) controleert eerst of de thread bestaat (**404**), dan de invoer (**422**, alle velden verplicht) en stuurt daarna de gewijzigde thread terug (**200**).
- `patch()` doet hetzelfde, maar controleert alleen de meegestuurde velden.
- `destroy()` verwijdert de thread. Was er niets om te verwijderen, dan **404**; anders **204 No Content**.

De `TopicController` en `ReplyController` werken precies hetzelfde. Alleen het model en de regels zijn anders:

app/Http/Controllers/TopicController.php

```
<?php
namespace App\Http\Controllers;

use App\Http\ApiResponse;
use App\Models\TopicModel;

class TopicController extends Controller
{
    // Regels voor de velden van een topic
    private const RULES = [
        'title' => 'required|string|max:100',
        'content' => 'required|string',
        'thread_id' => 'required|int',
        'user_id' => 'required|int',
    ];

    // GET /topics
    public function index(): void
    {
        ApiResponse::sendResponse(TopicModel::all());
    }

    // GET /topics/{id}
    public function show(int $id): void
    {
        $topic = TopicModel::find($id);

        if ($topic === null) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Topic {$id} bestaat niet.");
        }

        ApiResponse::sendResponse($topic);
    }

    // POST /topics
    public function create(array $request): void
    {
        $errors = $this->validate($request, self::RULES);

        if (!empty($errors)) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_UNPROCESSABLE, 'De invoer is niet geldig.', $errors);
        }

        $topic = TopicModel::create($request);

        header('Location: ' . ApiResponse::url('topics/' . $topic['id']));
        ApiResponse::sendResponse($topic, ApiResponse::HTTP_STATUS_CREATED);
    }

    // PUT /topics/{id} : alle velden zijn verplicht
    public function update(int $id, array $request): void
    {
        if (TopicModel::find($id) === null) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Topic {$id} bestaat niet.");
        }

        $errors = $this->validate($request, self::RULES);

        if (!empty($errors)) {
```

```

        ApiResponse::sendError(ApiResponse::HTTP_STATUS_UNPROCESSABLE, 'De invoer is niet
geldig.', $errors);
    }

    ApiResponse::sendResponse(TopicModel::update($id, $request));
}

// PATCH /topics/{id} : alleen de meegestuurde velden worden aangepast
public function patch(int $id, array $request): void
{
    if (TopicModel::find($id) === null) {
        ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Topic {$id} bestaat niet.");
    }

    $errors = $this->validate($request, self::RULES, true);

    if (!empty($errors)) {
        ApiResponse::sendError(ApiResponse::HTTP_STATUS_UNPROCESSABLE, 'De invoer is niet
geldig.', $errors);
    }

    ApiResponse::sendResponse(TopicModel::update($id, $request));
}

// DELETE /topics/{id}
public function destroy(int $id): void
{
    if (!TopicModel::delete($id)) {
        ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Topic {$id} bestaat niet.");
    }

    ApiResponse::sendResponse(null, ApiResponse::HTTP_STATUS_NO_CONTENT);
}
}

```

#### app/Http/Controllers/ReplyController.php

```

<?php
namespace App\Http\Controllers;

use App\Http\ApiResponse;
use App\Models\ReplyModel;

class ReplyController extends Controller
{
    // Regels voor de velden van een reply
    private const RULES = [
        'content' => 'required|string',
        'topic_id' => 'required|int',
        'user_id' => 'required|int',
    ];

    // GET /replies
    public function index(): void
    {
        ApiResponse::sendResponse(ReplyModel::all());
    }

    // GET /replies/{id}
    public function show(int $id): void
    {
        $reply = ReplyModel::find($id);

        if ($reply === null) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Reply {$id} bestaat niet.");
        }
    }
}

```

```

    }

    ApiResponse::sendResponse($reply);
}

// POST /replies
public function create(array $request): void
{
    $errors = $this->validate($request, self::RULES);

    if (!empty($errors)) {
        ApiResponse::sendError(ApiResponse::HTTP_STATUS_UNPROCESSABLE, 'De invoer is niet
geldig.', $errors);
    }

    $reply = ReplyModel::create($request);

    header('Location: ' . ApiResponse::url('replies/' . $reply['id']));
    ApiResponse::sendResponse($reply, ApiResponse::HTTP_STATUS_CREATED);
}

// PUT /replies/{id} : alle velden zijn verplicht
public function update(int $id, array $request): void
{
    if (ReplyModel::find($id) === null) {
        ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Reply {$id} bestaat niet.");
    }

    $errors = $this->validate($request, self::RULES);

    if (!empty($errors)) {
        ApiResponse::sendError(ApiResponse::HTTP_STATUS_UNPROCESSABLE, 'De invoer is niet
geldig.', $errors);
    }

    ApiResponse::sendResponse(ReplyModel::update($id, $request));
}

// PATCH /replies/{id} : alleen de meegestuurde velden worden aangepast
public function patch(int $id, array $request): void
{
    if (ReplyModel::find($id) === null) {
        ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Reply {$id} bestaat niet.");
    }

    $errors = $this->validate($request, self::RULES, true);

    if (!empty($errors)) {
        ApiResponse::sendError(ApiResponse::HTTP_STATUS_UNPROCESSABLE, 'De invoer is niet
geldig.', $errors);
    }

    ApiResponse::sendResponse(ReplyModel::update($id, $request));
}

// DELETE /replies/{id}
public function destroy(int $id): void
{
    if (!ReplyModel::delete($id)) {
        ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Reply {$id} bestaat niet.");
    }

    ApiResponse::sendResponse(null, ApiResponse::HTTP_STATUS_NO_CONTENT);
}
}

```

## Stap 7: de routetabel

Maak `app/Routes/routes.php` met de routetabel uit hoofdstuk 10. Elke controller-methode heeft nu een route.

## Stap 8: de RequestHandler, de router

Als laatste het onderdeel dat alles aan elkaar knoopt. Maak `app/Http/RequestHandler.php`:

`app/Http/RequestHandler.php`

```
<?php
namespace App\Http;

class RequestHandler
{
    // Constanten voor de request-methodes
    public const GET = 'GET';
    public const POST = 'POST';
    public const PUT = 'PUT';
    public const PATCH = 'PATCH';
    public const DELETE = 'DELETE';
    public const OPTIONS = 'OPTIONS';

    // Properties
    private array $routes;
    private string $requestMethod;
    private string $resource;
    private ?int $id = null;

    public function __construct(array $routes)
    {
        $this->routes = $routes;
        $this->requestMethod = strtolower($_SERVER['REQUEST_METHOD']);

        $this->parseURI();
    }

    // Haalt de resource en het id uit de URL (die .htaccess in $_GET heeft gezet)
    private function parseURI(): void
    {
        if (isset($_GET['resource'])) {
            $this->resource = strtolower($_GET['resource']);
        } else {
            $this->resource = '/';
        }

        if (isset($_GET['id'])) {
            if (!ctype_digit($_GET['id']) || (int) $_GET['id'] < 1) {
                ApiResponse::sendError(
                    ApiResponse::HTTP_STATUS_BAD_REQUEST,
                    'Het id moet een positief geheel getal zijn.'
                );
            }

            $this->id = (int) $_GET['id'];
        }
    }

    // Maakt de sleutel om in routes.php te zoeken: 'threads' of 'threads/{id}'
    private function getRouteKey(): string
    {
        if ($this->id === null) {
            return $this->resource;
        }
    }
}
```

```

        return $this->resource . '{id}';
    }

    // Zoekt de route bij de methode en de URL. Geeft null als die niet bestaat.
    private function findRoute(): ?array
    {
        return $this->routes[$this->requestMethod][$this->getRouteKey()] ?? null;
    }

    // Geeft de methodes die WEL bestaan voor deze URL (nodig voor een 405)
    private function getAllowedMethods(): array
    {
        $allowed = [];

        foreach ($this->routes as $method => $routesForMethod) {
            if (isset($routesForMethod[$this->getRouteKey()])) {
                $allowed[] = $method;
            }
        }

        return $allowed;
    }

    // Leest de data die de client in de body van het request heeft gestuurd
    private function getRequestData(): array
    {
        $contentType = $_SERVER['CONTENT_TYPE'] ?? '';
        $body = file_get_contents('php://input');

        // 1. JSON (de standaard voor API's)
        if (str_contains($contentType, 'application/json')) {
            if (trim($body) === '') {
                return [];
            }

            $data = json_decode($body, true);

            if (!is_array($data)) {
                ApiResponse::sendError(
                    ApiResponse::HTTP_STATUS_BAD_REQUEST,
                    'De body bevat geen geldige JSON.'
                );
            }

            return $data;
        }

        // 2. Formulierdata bij POST: die zet PHP zelf in $_POST
        if ($this->requestMethod === self::POST) {
            return $_POST;
        }

        // 3. Formulierdata bij PUT en PATCH: die lezen we zelf uit
        parse_str($body, $data);

        return $data;
    }

    public function handleRequest(): void
    {
        // 1. Een OPTIONS-request is een CORS-preflight: meteen beantwoorden
        if ($this->requestMethod === self::OPTIONS) {
            ApiResponse::sendCORSHeaders();
        }
    }

```

```

// 2. Bestaat de route?
$route = $this->findRoute();

if ($route === null) {
    $allowedMethods = $this->getAllowedMethods();

    // De URL bestaat wel, maar niet met deze methode
    if (count($allowedMethods) > 0) {
        header('Allow: ' . implode(', ', $allowedMethods));
        ApiResponse::sendError(
            ApiResponse::HTTP_STATUS_METHOD_NOT_ALLOWED,
            "De methode {$this->requestMethod} is niet toegestaan voor deze URL."
        );
    }

    // De URL bestaat helemaal niet
    ApiResponse::sendError(
        ApiResponse::HTTP_STATUS_NOT_FOUND,
        'Deze route bestaat niet.'
    );
}

// 3. Controller en methode uit de route halen
[$classname, $methodName] = $route;
$controller = new $classname();

// 4. De argumenten voor de controller-methode verzamelen
$arguments = [];

if ($this->id !== null) {
    $arguments[] = $this->id;
}

if (in_array($this->requestMethod, [self::POST, self::PUT, self::PATCH])) {
    $arguments[] = $this->getRequestData();
}

// 5. De controller-methode aanroepen, bijvoorbeeld $controller->show(5)
$controller->$methodName(...$arguments);
}
}

```

We lopen de belangrijkste methodes door.

`__construct()` en `parseURI()` lezen de methode van het request en de stukjes van de URL die `.htaccess` in `$_GET` heeft gezet. Is er een id meegegeven dat geen positief geheel getal is (zoals `/threads/abc`), dan stuurt de API meteen `400 Bad Request`. `ctype_digit()` controleert of een tekst alleen uit cijfers bestaat.

`getRouteKey()` maakt de sleutel om in de routetabel te zoeken: `threads` als er geen id is, en `threads/{id}` als er wel een id is.

`findRoute()` zoekt in de routetabel. `$this->routes['GET']['threads/{id}']` geeft bijvoorbeeld `[ThreadController::class, 'show']`. Bestaat de route niet, dan geeft `?? null` de waarde `null`.

`getAllowedMethods()` zoekt bij welke methodes de URL **wel** bestaat. Dat is nodig om het verschil te maken tussen `404` en `405`.

`getRequestData()` leest de body van het request:

- Stuurt de client JSON (`Content-Type: application/json`), dan leest hij de body met `php://input` en zet hem om met `json_decode()`. Is de JSON kapot, dan stuurt hij `400`.
- Stuurt de client formulierdata bij een **POST**, dan staat die al in `$_POST`.
- Bij **PUT** en **PATCH** met formulierdata vult PHP `$_POST` niet. Dan leest hij de body zelf en zet hem om met `parse_str()`.

`handleRequest()` is het hart van de router:

1. Is het een **OPTIONS**-request, dan wordt meteen de preflight beantwoord.
2. Zoek de route. Bestaat die niet, maar bestaat de URL wel met een andere methode, dan volgt `405` met een `Allow`-header. Bestaat de URL helemaal niet, dan `404`.
3. Haal de klassenaam en de naam van de methode uit de route. `[$classname, $methodName] = $route;` zet het eerste element in `$classname` en het tweede in `$methodName`. Daarna wordt met `new $classname()` een object van die klasse gemaakt.
4. Verzamel de argumenten: het id (als dat er is) en de body (bij **POST**, **PUT** en **PATCH**).
5. Roep de methode aan. De drie puntjes in `...$arguments` (de **spread-operator**) zetten de array om in losse argumenten. Een array `[5, ['title' => 'Hoi']]` wordt dus `$controller->patch(5, ['title' => 'Hoi'])`.

## Hoe werkt alles samen?

Laten we één request helemaal volgen. De client stuurt:

### Request

```
PATCH /forum-api/threads/5 HTTP/1.1
Host: localhost
Content-Type: application/json

{"title": "Nieuwe titel"}
```

1. **Apache** ontvangt het request. De `.htaccess` herschrijft `threads/5` naar `index.php?resource=threads&id=5`.
2. **index.php** laadt de autoloader en de routetabel en maakt een `RequestHandler`.
3. De `RequestHandler` leest de methode `PATCH`, de resource `threads` en het id `5`. De route-sleutel wordt `threads/{id}`.
4. In de routetabel staat bij `PATCH` en `threads/{id}`: `[ThreadController::class, 'patch']`.
5. De body wordt gelezen: `['title' => 'Nieuwe titel']`. De `RequestHandler` roept `$controller->patch(5, ['title' => 'Nieuwe titel'])` aan.
6. De `ThreadController` vraagt het `ThreadModel` of thread 5 bestaat. Dat doet een `SELECT` in de database.
7. De controller valideert de invoer met `$partial = true`: alleen `title` wordt gecontroleerd, en die is in orde.
8. Het `ThreadModel` doet een `UPDATE` met alleen `title` en `updated_at`, en haalt de thread daarna opnieuw op.
9. `ApiResponse` stuurt de headers, de statuscode `200` en de JSON met `meta` en `data` terug.

### Response

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
Access-Control-Expose-Headers: Location
Content-Type: application/json; charset=utf-8

{
  "meta": {
    "api_version": "1.0",
    "api_name": "forum_api",
    "status": 200,
    "status_message": "OK"
  },
  "data": {
    "id": 5,
    "title": "Nieuwe titel",
    "content": "Quidem est laborum adipisci aspernatur asperiores. ...",
    "user_id": 4,
    "created_at": "2018-01-30 09:47:15",
    "updated_at": "2026-09-25 14:12:03"
  }
}
```

## Samenvatting

- De API is verdeeld in lagen: router (`RequestHandler`), controllers, models, `Database` en `ApiResponse`.
- `index.php` is de voordeur en vangt onverwachte fouten op met een `500`.
- De models erven van de basisklasse `Model`; `static::` zorgt dat dezelfde code voor elke tabel werkt.
- `$fillable` en prepared statements beschermen tegen mass assignment en SQL-injectie.
- De controllers valideren de invoer en kiezen de juiste statuscode.
- De `RequestHandler` zoekt de route, maakt onderscheid tussen `404` en `405`, leest de body en roept de controller aan.

## Opdracht: Hoofdstuk 13

1. Bouw de forum-API na en controleer in je browser dat `http://localhost/forum-api/threads` alle threads laat zien.
2. Leg in je eigen woorden uit wat het verschil is tussen `self::` en `static::` in de klasse `Model`.
3. Stel dat `Model` geen `onlyFillable()` zou gebruiken. Wat gebeurt er dan als een client bij een PATCH de body `{"title": "Hoi", "created_at": "2000-01-01 00:00:00"}` stuurt? En wat als hij `{"title": "Hoi", "hacker": 1}` stuurt?
4. Voeg de endpoints `GET /users` en `GET /users/{id}` toe. Maak daarvoor een `UserModel` en een `UserController`. Let op: het wachtwoord en het e-mailadres mogen **nooit** in de response staan! Tip: geef `UserModel` eigen versies van `all()` en `find()` met een `SELECT` van alleen de kolommen die wel mogen.
5. Als je een topic aanmaakt met een `thread_id` die niet bestaat, wordt de topic toch opgeslagen. Pas `TopicController::create()` aan, zodat de API dan `422` stuurt met bij `details` de melding *Thread 999 bestaat niet*.
6. **Uitdaging:** maak het endpoint `GET /threads/{id}/topics`, dat alle topics van één thread teruggeeft. Bestaat de thread niet, dan geeft het `404`. Je moet hiervoor de `.htaccess`, de `RequestHandler`, de routetabel, de klasse `Model` en de `TopicController` aanpassen.

## 14. De API testen met Postman

### Waarom testen?

Een API heeft geen scherm waarop je kunt klikken. Om te weten of hij goed werkt, moet je requests sturen en de responses controleren. Niet alleen of het gelukt is, maar ook of je bij fouten de juiste statuscode en melding krijgt. In dit hoofdstuk zet je in Postman een verzameling requests klaar, zodat je je hele API steeds opnieuw kunt testen.

### Een collection en een variabele

In Postman kun je requests bewaren in een **collection**: een map met requests die bij elkaar horen.

1. Klik links op **Collections** en daarna op het plusje. Noem de collection *Forum API*.
2. Klik op de naam van je collection en kies het tabblad **Variables**.
3. Maak een variabele `base_url` met als waarde `http://localhost/forum-api`. Sla op met Ctrl + S.

In je requests kun je nu `{{base_url}}/threads` typen in plaats van de hele URL. Verhuis je je API later naar een andere map of server, dan hoeft je alleen de variabele aan te passen.

### Requests maken

Klik bij je collection op de drie puntjes en kies **Add request**. Geef het een duidelijke naam, zoals *Alle threads ophalen*, kies de methode en typ de URL. Klik op **Save** om het te bewaren.

Voor POST, PUT en PATCH moet je een body meesturen:

- **JSON**: kies bij **Body** de optie **raw** en rechts **JSON**. Typ je JSON in het tekstvak. Postman zet de header `Content-Type: application/json` automatisch.
- **Formulierdata**: kies **x-www-form-urlencoded** en vul de velden in als *Key* en *Value*.

#### ⚠ Let op

Kies bij PUT en PATCH niet voor **form-data** (dat is `multipart/form-data`). Dat formaat kan onze API bij PUT en PATCH niet lezen. Gebruik JSON of `x-www-form-urlencoded`.

## Een testlijst

Maak voor elke regel in deze tabel een request in je collection en controleer of je de verwachte statuscode krijgt. Doe ze in deze volgorde, want sommige requests bouwen voort op eerdere.

| #  | Request                                            | Body                                                             | Verwacht                                  |
|----|----------------------------------------------------|------------------------------------------------------------------|-------------------------------------------|
| 1  | GET {{base_url}}/threads                           |                                                                  | 200, lijst met count                      |
| 2  | GET {{base_url}}/threads/1                         |                                                                  | 200, één thread                           |
| 3  | GET {{base_url}}/threads/999                       |                                                                  | 404                                       |
| 4  | GET {{base_url}}/threads/abc                       |                                                                  | 400                                       |
| 5  | GET {{base_url}}/bestaatniet                       |                                                                  | 404                                       |
| 6  | DELETE {{base_url}}/threads                        |                                                                  | 405, met header Allow: GET, POST          |
| 7  | POST {{base_url}}/threads                          | {"title": "Testthread",<br>"content": "Hallo!",<br>"user_id": 1} | 201, met header Location                  |
| 8  | POST {{base_url}}/threads                          | {"title": ""}                                                    | 422, met details                          |
| 9  | POST {{base_url}}/threads                          | {"title": "kapot"}                                               | 400                                       |
| 10 | PUT {{base_url}}/threads/{id}                      | alleen {"title": "Nieuw"}                                        | 422                                       |
| 11 | PUT {{base_url}}/threads/{id}                      | alle drie de velden                                              | 200                                       |
| 12 | PATCH {{base_url}}/threads/{id}                    | {"content": "Alleen dit is anders"}                              | 200                                       |
| 13 | PATCH {{base_url}}/threads/{id}                    | {"kleur": "rood"}                                                | 422                                       |
| 14 | DELETE {{base_url}}/threads/{id}                   |                                                                  | 204, geen body                            |
| 15 | DELETE {{base_url}}/threads/{id}<br>(nog een keer) |                                                                  | 404                                       |
| 16 | OPTIONS {{base_url}}/threads/1                     |                                                                  | 204, met Access-Control-Allow-...-headers |

Bij `{id}` gebruik je het id van de thread die je bij test 7 hebt aangemaakt. Dat staat in de response en in de `Location`-header.

### 💡 Tip

Kijk bij elk request niet alleen naar de body, maar ook naar het tabblad **Headers** van de response. Zie je bij test 7 de `Location`-header? En bij test 6 de `Allow`-header?

## Automatische tests in Postman

Je kunt Postman ook zelf laten controleren of de response klopt. Open het request van test 7 en klik op het tabblad **Scripts** en daarna op **Post-response**. (In oudere versies van Postman heet dit tabblad **Tests**.) Typ daar:

### Postman: Post-response script bij test 7

```
pm.test('Statuscode is 201', () => {
  pm.response.to.have.status(201)
})

pm.test('Response heeft meta en data', () => {
  const json = pm.response.json()
  pm.expect(json).to.have.property('meta')
  pm.expect(json).to.have.property('data')
})

pm.test('Location-header is aanwezig', () => {
  pm.response.to.have.header('Location')
})

// Het id van de nieuwe thread bewaren voor de volgende requests
pm.collectionVariables.set('thread_id', pm.response.json().data.id)
```

Stuur het request. Onderaan, bij **Test Results**, zie je nu of de tests geslaagd zijn. De laatste regel bewaart het id van de nieuwe thread in de variabele `thread_id`. In de requests 10 tot en met 15 kun je dan `{{base_url}}/threads/{{thread_id}}` gebruiken.

Met de **Collection Runner** voer je alle requests van je collection in één keer uit: klik bij je collection op de drie puntjes en kies **Run collection**. Zo test je na elke wijziging in je code in een paar seconden je hele API.

### 📌 Samenvatting

- Bewaar je requests in een **collection** en gebruik een variabele `{{base_url}}`.
- Test niet alleen wat goed moet gaan, maar ook of fouten de juiste statuscode geven.
- Kijk ook naar de response-headers, zoals `Location` en `Allow`.
- Met **Post-response scripts** laat je Postman de responses automatisch controleren.

**Opdracht: Hoofdstuk 14**

1. Maak de collection *Forum API* met de variabele `base_url` en alle 16 requests uit de testlijst. Krijg je overal de verwachte statuscode?
2. Voeg het Post-response script toe aan test 7 en gebruik `{{thread_id}}` in de requests 10 tot en met 15. Voer de hele collection uit met de Collection Runner.
3. Schrijf een Post-response script voor test 3 dat controleert of de statuscode `404` is en of de response een `error` met een `message` bevat.
4. Maak een testlijst (zoals de tabel hierboven) voor de endpoints van **replies**, met minstens zes tests. Denk aan goede én foute requests.

## 15. Afronding en verder leren

### Wat heb je geleerd?

Kijk eens hoe ver je bent gekomen! Je weet nu:

- wat een API is, en hoe client en server via **HTTP** met requests en responses praten;
- welke **HTTP-methodes** er zijn en wanneer je GET, POST, PUT, PATCH, DELETE en OPTIONS gebruikt;
- wat **headers** zijn en hoe je ze in PHP verstuurt en uitleest;
- hoe **JSON** werkt en hoe je het in PHP maakt en leest;
- welke **statuscodes** er zijn en welke je wanneer gebruikt;
- wat een **RESTful API** is en hoe je goede URL's ontwerpt;
- hoe je responses een vaste, duidelijke **structuur** geeft;
- hoe **.htaccess** en een **routetabel** URL's naar de juiste code sturen;
- hoe **Composer**, **namespaces** en **PSR-4** je klassen automatisch laden;
- wat **CORS** is en hoe je het instelt;
- en hoe je dat allemaal combineert tot een echte, geteste REST API.

### Hoe ga je verder?

Onze forum-API is een mooie basis, maar een echte API heeft meer nodig. Dit zijn goede volgende stappen:

**Authenticatie.** Nu kan iedereen alles aanmaken, wijzigen en verwijderen, en de client geeft zelf een `user_id` mee. Een echte API wil weten **wie** het request stuurt. Meestal logt een gebruiker één keer in en krijgt hij een **token**: een lange, geheime code. Die stuurt de client daarna bij elk request mee in de header `Authorization: Bearer <token>`. Zonder geldig token stuurt de API **401**; met een token van iemand die iets niet mag, **403**. De lege `LoginController` en `RegisterController` uit het originele project waren daarvoor bedoeld.

**Veilige wachtwoorden.** De wachtwoorden in de tabel `users` zijn opgeslagen met **MD5**. Dat is tegenwoordig niet veilig meer: MD5 is zo snel dat computers miljarden wachtwoorden per seconde kunnen uitproberen. Gebruik in PHP altijd `password_hash()` om een wachtwoord op te slaan en `password_verify()` om het te controleren.

**Instellingen buiten je code.** De gegevens van de database staan nu in `Database.php`. Beter is het om zulke instellingen in een apart bestand `.env` te zetten, dat je niet deelt op GitHub. Met het Composer-pakket `vlucas/phpdotenv` lees je dat bestand uit.

**Pagina's, filteren en sorteren.** Bij duizenden replies wil je niet alles in één keer terugsturen. Met de query string kun je pagina's (`?page=2&limit=20`), filters (`?user_id=3`) en sorteren (`?sort=-created_at`) toevoegen.

**Documentatie.** Andere programmeurs moeten weten hoe je API werkt. Veel API's beschrijven hun endpoints met de standaard **OpenAPI** (ook bekend als *Swagger*). Daarmee kun je automatisch een mooie website met documentatie maken.

**Live zetten.** Op een echte server gebruik je altijd **HTTPS**, zet je `display_errors` uit, gebruik je een aparte databasegebruiker met een sterk wachtwoord, en bescherm je je API tegen te veel requests (`429 Too Many Requests`).

**Een framework.** Nu je weet hoe een API van binnen werkt, kun je een framework gaan gebruiken, zoals **Laravel** of **Slim**. Je zult veel herkennen: routes, controllers, models, validatie en responses. Het framework doet veel werk voor je, maar jij snapt nu wat het doet!

Succes, en vooral: veel plezier met programmeren!

# Bijlage A: Antwoorden

---

Hieronder staan de antwoorden op de opdrachten. Bij programmeeropdrachten zijn er vaak meerdere goede oplossingen. Werkt jouw code en doet hij wat de opdracht vraagt? Dan is je antwoord ook goed.

## Hoofdstuk 1

1. Bijvoorbeeld: een API is als de balie van een bibliotheek. Je loopt niet zelf het magazijn in, maar vraagt aan de balie om een boek. De medewerker haalt het op en geeft het aan je, volgens vaste regels (je hebt een pasje nodig, je kunt alleen boeken lenen die in de catalogus staan).
2. Bijvoorbeeld: een weer-app (temperatuur, neerslag), Spotify (nummers, afspeellijsten), Instagram (foto's, likes, reacties), Google Maps (kaarten, routes), een bank-app (saldo, betalingen).
3. Een website stuurt HTML: een opgemaakte pagina voor mensen. Een API stuurt alleen gegevens (meestal JSON) voor een ander programma, dat zelf bepaalt hoe het de gegevens toont.
4. De API hoeft maar één keer gebouwd en onderhouden te worden. Alle apps krijgen dezelfde gegevens en werken op dezelfde manier. Een wijziging in de API geldt meteen voor alle apps.
5. Bijvoorbeeld: producten, categorieën, klanten, bestellingen, winkelwagens, betalingen, reviews, voorraad.

## Hoofdstuk 2

1. HTTP staat voor HyperText Transfer Protocol. Een protocol is een vaste set afspraken over hoe computers met elkaar communiceren.
2. Elk request staat op zichzelf. De server onthoudt niets van eerdere requests. Moet de server iets weten, zoals wie je bent, dan moet dat bij elk request opnieuw worden meegestuurd.
3. Scheme: `https`. Host: `api.weer.nl`. Path: `/v1/steden/groningen`. Query string: `dagen=3&eenheid=celsius`.
4. Een request: request-regel (methode, pad, versie), headers en eventueel een body. Een response: statusregel (versie, statuscode, tekst), headers en meestal een body.
5. Dit verschilt per website. `200` betekent gelukt. `304 Not Modified` betekent dat de browser het bestand al had en het niet opnieuw hoeft te downloaden. `404` betekent niet gevonden.

## Hoofdstuk 3

1. Als het gelukt is, zijn Apache en MySQL groen in het XAMPP Control Panel.
2. De statuscode is **200 OK**. De header **Content-Type** heeft de waarde **application/json**.
3. Bijvoorbeeld:

api-test/hallo.php

```
<?php
header('Content-Type: application/json');

$antwoord = [
    'bericht'      => 'Hallo! Mijn eerste API werkt.',
    'tijd'         => date('H:i:s'),
    'naam'         => 'Sam',
    'lievelingsgetal' => 7,
];

echo json_encode($antwoord);
```

4. In de originele database staan 22 rijen in **threads** en 60 rijen in **topics**. (Heb je al dingen toegevoegd of verwijderd, dan kan het anders zijn.)
5. In de tabel **replies**, in de kolom **topic\_id**.

## Hoofdstuk 4

1. a. **POST** b. **GET** c. **PATCH** d. **DELETE** e. **PUT**
2. Met PUT vervang je een ding helemaal en stuur je alle velden mee. Met PATCH wijzig je alleen de velden die je meestuurt. Voorbeeld: je profiel. Met PUT stuur je je naam, e-mail, woonplaats én foto opnieuw op. Met PATCH stuur je alleen je nieuwe woonplaats op.
3. Elke POST maakt iets nieuws aan: tien keer dezelfde POST geeft tien nieuwe dingen. Na één DELETE is het ding weg, en na tien keer DELETE is het nog steeds (gewoon) weg: het resultaat is hetzelfde.
4. Bij **OPTIONS** krijg je **{"methode":"OPTIONS","bericht":"Deze methode ken ik niet."}**, omdat die methode niet in de **switch** staat.
5. Bijvoorbeeld:

api-test/methode.php (deel)

```
case 'GET':
    $bericht = 'Je wilt iets ophalen.';
    $threads = [
        ['id' => 1, 'title' => 'Games'],
        ['id' => 2, 'title' => 'Huiswerkhulp'],
        ['id' => 3, 'title' => 'Muziek'],
    ];
    break;
```

En onderaan:

api-test/methode.php (deel)

```
$antwoord = [
    'methode' => $methode,
    'bericht' => $bericht,
];

if (isset($threads)) {
    $antwoord['threads'] = $threads;
}

echo json_encode($antwoord);
```

## Hoofdstuk 5

1. Met **Accept** zegt de client in welk formaat hij de **response** graag wil krijgen. Met **Content-Type** zegt de client in welk formaat de **body van zijn eigen request** is.
2. In `$_SERVER['HTTP_AUTHORIZATION']`.
3. In de response zie je bij **alle\_headers** de regel `"X-Klas": "4B"`.
4. Er zijn twee fouten. Er staat een lege regel vóór `<?php`, en er wordt al iets ge-**echod** vóór `header()`. In beide gevallen is de body al begonnen en is het te laat voor headers. Goed is:

Verbeterd

```
<?php
header('Content-Type: application/json');
echo json_encode('Hallo');
```

5. Bijvoorbeeld:

api-test/hallo.php

```
<?php
$accept = $_SERVER['HTTP_ACCEPT'] ?? '';

if (!str_contains($accept, 'application/json')) {
    header('Content-Type: text/plain; charset=utf-8');
    echo 'Ik spreek alleen JSON.';
    exit;
}

header('Content-Type: application/json');

echo json_encode([
    'bericht' => 'Hallo! Mijn eerste API werkt.',
    'tijd'    => date('H:i:s'),
]);
```

Test het in Postman door bij **Headers** de header **Accept** aan te passen.

## Hoofdstuk 6

1. `'titel'` staat tussen enkele aanhalingstekens, `aantal` staat niet tussen aanhalingstekens, en na `"console"` staat een komma na het laatste item. Goed is: `{"titel": "Games", "aantal": 3, "tags": ["pc", "console"]}`
2. Bijvoorbeeld:

Een reply

```
{
  "id": 12,
  "content": "Goed idee!",
  "topic_id": 4,
  "user_id": 7,
  "updated_at": null
}
```

3. `['a', 'b']` is een lijst en wordt de JSON-array `["a","b"]`. `['x' => 'a', 'y' => 'b']` is een associatieve array en wordt het JSON-object `{"x":"a","y":"b"}`.
4. Bij geldige JSON krijg je de gegevens terug met `aantal_velden`. Bij ongeldige JSON krijg je `{"fout":"Dit is geen geldige JSON."}`.
5. PHP vult `$_POST` alleen bij formulierdata (`application/x-www-form-urlencoded` en `multipart/form-data`). JSON moet je zelf uit de body lezen met `file_get_contents('php://input')` en omzetten met `json_decode()`.

## Hoofdstuk 7

1. Bij een 4xx heeft de client iets fout gedaan; de client moet zijn request aanpassen (andere URL, andere gegevens, inloggen). Opnieuw hetzelfde proberen heeft geen zin. Bij een 5xx ligt het probleem bij de server; de client kan het later opnieuw proberen.
2. a. 201 b. 404 c. 400 d. 422 e. 403 f. 500 (of 503) g. 405 h. 204
3. De statuscode 200 zegt dat het gelukt is, terwijl het mislukt is. De client moet dan elke body lezen om te weten of het goed ging. Beter: statuscode 404, en in de body een vaste foutstructuur, bijvoorbeeld `{"error": {"message": "Niet gevonden"}}`.
4. De echte foutmelding kan geheime informatie bevatten, zoals namen van tabellen en kolommen, de SQL-query of bestandspaden op de server. Een hacker kan daarmee zien hoe je systeem werkt en makkelijker een aanval bedenken. Schrijf de echte fout in een logbestand en geef de client een algemene melding.
5. Bijvoorbeeld, direct na de controle op `null`:

api-test/status.php (deel)

```
// Het id moet uit alleen cijfers bestaan
if (!ctype_digit($id)) {
    http_response_code(400);
    echo json_encode(['fout' => 'Het id moet een getal zijn.']);
    exit;
}
```

## Hoofdstuk 8

1. Een RESTful API houdt zich aan de afspraken van REST. Bijvoorbeeld: alles is een resource met een eigen URL; de URL is een zelfstandig naamwoord en de methode het werkwoord; stateless; gebruik van de standaard HTTP-methodes, statuscodes en headers; client en server zijn gescheiden.
2. a. `GET /replies` b. `POST /topics` c. `GET /threads/3` d. `DELETE /replies/8` e. `GET /users/12`
3. Bijvoorbeeld:

| Methode | URL                      | Actie                          | Succes |
|---------|--------------------------|--------------------------------|--------|
| GET     | <code>/books</code>      | Alle boeken ophalen            | 200    |
| GET     | <code>/books/{id}</code> | Eén boek ophalen               | 200    |
| POST    | <code>/books</code>      | Een boek toevoegen             | 201    |
| PUT     | <code>/books/{id}</code> | Een boek helemaal vervangen    | 200    |
| PATCH   | <code>/books/{id}</code> | Een boek gedeeltelijk wijzigen | 200    |
| DELETE  | <code>/books/{id}</code> | Een boek verwijderen           | 204    |

4. `GET /topics/4/replies`
5. De nieuwe thread heeft nog geen id; dat bepaalt de server. Je voegt iets toe aan de **collectie** `/threads`. Daarna vertelt de server in de `Location`-header waar het nieuwe item staat.

## Hoofdstuk 9

1. Dan kan de client met één stukje code elke response lezen. Bij een wisselende structuur moet de programmeur voor elk endpoint andere code schrijven, en ontstaan er sneller fouten.
2. Bijvoorbeeld: de namen gebruiken door elkaar PascalCase (**Reply**, **Content**), hoofdletters (**ID**), camelCase (**topicId**) en snake\_case (**user\_id**); getallen staan als tekst ("**3**", "**2**"); er staat een (versleuteld) wachtwoord in de response; de status staat als tekst "**gelukt**" in plaats van een vaste **meta** met een statuscode; de gegevens staan onder **Reply** in plaats van onder **data**.
3. Een lege lijst is geen fout, dus:

GET /topics?title=bestaatniet

```
{
  "meta": {
    "api_version": "1.0",
    "api_name": "forum_api",
    "status": 200,
    "status_message": "OK",
    "count": 0
  },
  "data": []
}
```

4. Het veld **content** is meegestuurd, maar leeg, en daardoor is de invoer niet geldig:

PATCH /replies/3 met een lege content

```
{
  "meta": {
    "api_version": "1.0",
    "api_name": "forum_api",
    "status": 422,
    "status_message": "Unprocessable Content"
  },
  "error": {
    "message": "De invoer is niet geldig.",
    "details": {
      "content": "Het veld content is verplicht."
    }
  }
}
```

5. De server vult zelf dingen in, zoals het **id** en de datums. Zo krijgt de client die meteen, en hoeft hij niet nog een extra GET-request te doen.

## Hoofdstuk 10

1. Een front controller is één bestand (bij ons `index.php`) waar alle requests binnenkomen. Het voordeel is dat alle requests op één plek beginnen: je kunt op één plek de routes zoeken, fouten opvangen en headers zetten.
2. `[L]`: dit is de laatste regel, stop met herschrijven. `[QSA]`: plak de query string van de client erachter. `[F]`: stuur `403 Forbidden`.
3. a. `index.php?resource=replies` b. `index.php?resource=topics&id=12` c. `index.php?resource=threads&id=3&pagina=2` d. `403 Forbidden`
4. De twee bestanden:

### api-test/dieren.php

```
<?php
header('Content-Type: application/json');

echo json_encode(['dier_id' => (int) $_GET['id']]);
```

### api-test/.htaccess

```
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /api-test/

  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteCond %{REQUEST_FILENAME} !-d
  RewriteRule ^dieren/([0-9]+)/?$ dieren.php?id=$1 [L,QSA]
</IfModule>
```

Omdat het patroon `[0-9]+` is, wordt `dieren/abc` niet herschreven en krijg je de 404-pagina van Apache.

5. De regel `RewriteBase /forum-api/` moet `RewriteBase /mijn-api/` worden.

## Hoofdstuk 11

1. Met namespaces kun je klassen met dezelfde naam uit elkaar houden, en ordenen in groepen. Daarnaast zijn ze nodig voor autoloading volgens PSR-4, omdat de autoloader uit de namespace afleidt in welke map een klasse staat.
2. `app/Http/Controllers/UserController.php` en `app/Services/Mail/Mailer.php`.
3. `namespace App\Models;`
4. Je ziet iets zoals *Composer version 2.8.4*.
5. De klassenaam is met een kleine letter geschreven (`threadModel`), terwijl de klasse en het bestand `ThreadModel` heten. Op Windows gaat dat soms toevallig goed, maar op Linux niet. Waarschijnlijk staat er een tikfout in een `use`-regel of in de routetabel.
6. De map `vendor` wordt door Composer gemaakt en kan heel groot worden. Iedereen kan hem zelf maken met `composer install`, dankzij `composer.json` (en `composer.lock`).

## Hoofdstuk 12

1. Een origin is de combinatie van scheme, host en poort. `http://localhost:8080` en `http://localhost:3000` hebben een andere poort, dus het zijn **verschillende** origins.
2. CORS (de same-origin policy) wordt alleen door de browser gecontroleerd. Postman is geen browser en trekt zich er niets van aan.
3. b en c veroorzaken een preflight. a is een simpel GET-request, en d is een simpel POST-request met gewone formulierdata.
4. Voeg bovenaan `hallo.php`, direct na `<?php`, deze regel toe:

`api-test/hallo.php` (deel)

```
header('Access-Control-Allow-Origin: *');
```

5. Bij een POST met JSON doet de browser eerst een preflight. Die moet `echo.php` ook beantwoorden. Zet bovenaan, direct na `<?php`:

`api-test/echo.php` (deel)

```
// CORS: elke website mag deze API gebruiken
header('Access-Control-Allow-Origin: *');

// De preflight beantwoorden
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {
    header('Access-Control-Allow-Methods: POST, OPTIONS');
    header('Access-Control-Allow-Headers: Content-Type');
    http_response_code(204);
    exit;
}
```

6. Standaard mag JavaScript op een andere origin de header `Location` niet lezen. Met `Access-Control-Expose-Headers: Location` geeft de API toestemming, zodat een app na een POST kan zien waar de nieuwe thread te vinden is.

## Hoofdstuk 13

1. Als alles goed is, zie je een response met `meta` (met `count`) en `data` met alle threads.
2. `self::` verwijst altijd naar de klasse waarin de code staat, dus `Model`. `static::` verwijst naar de klasse waarop je de methode aanroept, zoals `ThreadModel` of `ReplyModel`. Daardoor gebruikt `static::$table` bij `ThreadModel::all()` de tabel `threads` en bij `ReplyModel::all()` de tabel `replies`.
3. Bij de eerste body zou ook de kolom `created_at` worden aangepast, terwijl alleen de server die hoort in te vullen. Een client kan zo gegevens veranderen die hij niet mag veranderen (mass assignment). Bij de tweede body zou `hacker` als kolomnaam in de SQL komen. Die kolom bestaat niet, dus de database geeft een fout en de client krijgt `500`. Nog erger: een slimme aanvaller zou via een kolomnaam zelf SQL-code in de query kunnen smokkelen.
4. Het model en de controller:

app/Models/UserModel.php

```
<?php
namespace App\Models;

class UserModel extends Model
{
    protected static string $table = 'users';
    protected static array $fillable = [ 'username', 'email', 'avatar' ];

    // Alleen deze kolommen mogen naar buiten. Nooit het wachtwoord!
    private const PUBLIC_COLUMNS = '`id`, `username`, `avatar`, `created_at`';

    public static function all(): array
    {
        self::db()->query('SELECT ' . self::PUBLIC_COLUMNS . ' FROM `users`');

        return self::db()->getAll();
    }

    public static function find(int $id): ?array
    {
        self::db()->query(
            'SELECT ' . self::PUBLIC_COLUMNS . ' FROM `users` WHERE `id` = :id',
            [ 'id' => $id ]
        );

        return self::db()->get();
    }
}
```

## app/Http/Controllers/UserController.php

```
<?php
namespace App\Http\Controllers;

use App\Http\ApiResponse;
use App\Models\UserModel;

class UserController extends Controller
{
    // GET /users
    public function index(): void
    {
        ApiResponse::sendResponse(UserModel::all());
    }

    // GET /users/{id}
    public function show(int $id): void
    {
        $user = UserModel::find($id);

        if ($user === null) {
            ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "User {$id} bestaat niet.");
        }

        ApiResponse::sendResponse($user);
    }
}
```

En in `routes.php` bovenaan `use App\Http\Controllers\UserController;` en bij GET:

## app/Routes/routes.php (deel)

```
'users' => [ UserController::class, 'index' ],
'users/{id}' => [ UserController::class, 'show' ],
```

5. Zet in `TopicController.php` bovenaan `use App\Models\ThreadModel;` en in `create()`, na de validatie en vóór `TopicModel::create()`:

## app/Http/Controllers/TopicController.php (deel)

```
// Bestaat de thread waar deze topic bij hoort?
if (ThreadModel::find((int) $request['thread_id']) === null) {
    ApiResponse::sendError(
        ApiResponse::HTTP_STATUS_UNPROCESSABLE,
        'De invoer is niet geldig.',
        [ 'thread_id' => "Thread {$request['thread_id']} bestaat niet." ]
    );
}
```

Voor PUT en PATCH kun je dezelfde controle toevoegen in `update()` en `patch()` (bij `patch()` alleen als `thread_id` is meegestuurd).

6. Er zijn vijf aanpassingen nodig.

Een extra regel in de `.htaccess`, **boven** de regel voor `threads/1`:

#### `.htaccess` (deel)

```
# http://localhost/forum-api/threads/1/topics
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^([^\s]+)/([^\s]+)/([^\s]+)/?$ index.php?resource=$1&id=$2&relation=$3 [L,QSA]
```

In de `RequestHandler` een nieuwe property, het uitlezen ervan in `parseURI()`, en een aangepaste `getRouteKey()`:

#### `app/Http/RequestHandler.php` (deel)

```
private ?string $relation = null;

// In parseURI(), onderaan:
if (isset($_GET['relation'])) {
    $this->relation = strtolower($_GET['relation']);
}

// getRouteKey() wordt:
private function getRouteKey(): string
{
    if ($this->id === null) {
        return $this->resource;
    }

    if ($this->relation !== null) {
        return $this->resource . '/{id}/' . $this->relation;
    }

    return $this->resource . '/{id}';
}
```

Een nieuwe route bij `GET` in `routes.php`:

#### `app/Routes/routes.php` (deel)

```
'threads/{id}/topics' => [ TopicController::class, 'byThread' ],
```

Een nieuwe methode `where()` in `Model.php`. De kolomnaam komt alleen uit onze eigen code, nooit van de client:

#### `app/Models/Model.php` (deel)

```
// Alle rijen waarbij een kolom een bepaalde waarde heeft
public static function where(string $column, int|string $value): array
{
    $table = static::$table;

    self::db()->query("SELECT * FROM `{$table}` WHERE `{$column}` = :value", [ 'value' => $value ]);

    return self::db()->getAll();
}
```

En een nieuwe methode in `TopicController.php` (met `use App\Models\ThreadModel;` bovenaan):

`app/Http/Controllers/TopicController.php` (deel)

```
// GET /threads/{id}/topics
public function byThread(int $threadId): void
{
    if (ThreadModel::find($threadId) === null) {
        ApiResponse::sendError(ApiResponse::HTTP_STATUS_NOT_FOUND, "Thread {$threadId} bestaat
niet.");
    }

    ApiResponse::sendResponse(TopicModel::where('thread_id', $threadId));
}
```

`GET /threads/1/topics` geeft nu de drie topics van thread 1, en `GET /threads/99/topics` geeft `404`.

## Hoofdstuk 14

1. Als je API goed gebouwd is, krijg je bij elke test de verwachte statuscode.
2. In de Collection Runner zie je bij elk request of het gelukt is, en bij test 7 dat alle drie de tests geslaagd zijn.
3. Bijvoorbeeld:

### Postman: Post-response script bij test 3

```
pm.test('Statuscode is 404', () => {
  pm.response.to.have.status(404)
})

pm.test('Response heeft een error met een message', () => {
  const json = pm.response.json()
  pm.expect(json).to.have.property('error')
  pm.expect(json.error).to.have.property('message')
})
```

4. Bijvoorbeeld:

| # | Request                          | Body                                              | Verwacht |
|---|----------------------------------|---------------------------------------------------|----------|
| 1 | GET {{base_url}}/replies         |                                                   | 200      |
| 2 | GET {{base_url}}/replies/1       |                                                   | 200      |
| 3 | GET {{base_url}}/replies/99999   |                                                   | 404      |
| 4 | POST {{base_url}}/replies        | {"content": "Leuk!", "topic_id": 2, "user_id": 3} | 201      |
| 5 | POST {{base_url}}/replies        | {"content": "Leuk!"}                              | 422      |
| 6 | PATCH {{base_url}}/replies/{id}  | {"content": "Aangepast"}                          | 200      |
| 7 | PUT {{base_url}}/replies/{id}    | {"content": "Alleen dit"}                         | 422      |
| 8 | DELETE {{base_url}}/replies/{id} |                                                   | 204      |
| 9 | POST {{base_url}}/replies/5      | {"content": "Hoi"}                                | 405      |

## Bijlage B: Begrippenlijst

| Begrip                   | Betekenis                                                                                                     |
|--------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>.htaccess</code>   | Instellingenbestand voor Apache, per map. Wij gebruiken het voor URL rewriting.                               |
| API                      | Application Programming Interface: een vaste manier waarop programma's met elkaar praten.                     |
| Apache                   | De webserver in XAMPP.                                                                                        |
| Autoloading              | Klassen automatisch inladen zodra ze nodig zijn.                                                              |
| Backend                  | Alles wat achter de schermen gebeurt: server, database, API.                                                  |
| Body                     | Het deel van een request of response met de eigenlijke gegevens.                                              |
| Client                   | Het programma dat een request stuurt.                                                                         |
| Collection (Postman)     | Een map met bewaarde requests in Postman.                                                                     |
| Composer                 | Pakketbeheerder voor PHP; maakt ook de autoloader.                                                            |
| Controller               | Klasse die bepaalt wat er bij een endpoint gebeurt.                                                           |
| CORS                     | Cross-Origin Resource Sharing: toestemming van de server om een API vanaf een andere origin te gebruiken.     |
| CRUD                     | Create, Read, Update, Delete: de vier basisacties op gegevens.                                                |
| Endpoint                 | Eén combinatie van een HTTP-methode en een URL.                                                               |
| Envelop                  | Vaste verpakking van een response, bij ons met <code>meta</code> en <code>data</code> of <code>error</code> . |
| Foreign key              | Kolom die verwijst naar een rij in een andere tabel, zoals <code>thread_id</code> .                           |
| Front controller         | Eén bestand waar alle requests binnenkomen, bij ons <code>index.php</code> .                                  |
| Frontend                 | Alles wat de gebruiker ziet: de website of app.                                                               |
| Header                   | Extra informatie bij een request of response: <code>Naam: waarde</code> .                                     |
| HTTP                     | HyperText Transfer Protocol: het protocol van het web.                                                        |
| HTTPS                    | HTTP met een beveiligde, versleutelde verbinding.                                                             |
| Idempotent               | Vaker uitvoeren geeft hetzelfde resultaat als één keer uitvoeren.                                             |
| JSON                     | JavaScript Object Notation: tekstformaat voor gegevens.                                                       |
| Mass assignment          | Een client die velden aanpast die hij niet mag aanpassen. Bescherming: <code>\$fillable</code> .              |
| Media type               | Soort gegevens, zoals <code>application/json</code> . Staat in <code>Content-Type</code> .                    |
| Methode                  | Het werkwoord van een request: GET, POST, PUT, PATCH, DELETE, OPTIONS.                                        |
| Model                    | Klasse die met de database praat voor één tabel.                                                              |
| <code>mod_rewrite</code> | Apache-module om URL's te herschrijven.                                                                       |
| Namespace                | Soort map of achternaam voor klassen, zoals <code>App\Models</code> .                                         |
| Origin                   | Combinatie van scheme, host en poort.                                                                         |
| PDO                      | Ingebouwde PHP-klasse om met databases te werken.                                                             |

| Begrip              | Betekenis                                                                                          |
|---------------------|----------------------------------------------------------------------------------------------------|
| Preflight           | Automatisch OPTIONS-request van de browser om toestemming te vragen (CORS).                        |
| Prepared statement  | SQL-query waarbij de waarden apart worden meegestuurd; voorkomt SQL-injectie.                      |
| Protocol            | Vaste set afspraken over hoe computers communiceren.                                               |
| PSR-4               | Afspraak over waar klassen staan: namespace = mappen, klassenaam = bestandsnaam.                   |
| Query string        | Het deel van een URL na het <code>?</code> , zoals <code>?sort=nieuw</code> .                      |
| Request             | Het verzoek van de client aan de server.                                                           |
| Resource            | Een 'ding' waar een API over gaat, zoals een thread.                                               |
| Response            | Het antwoord van de server.                                                                        |
| REST / RESTful      | Set afspraken voor het ontwerpen van API's; een API die zich eraan houdt.                          |
| Route               | Koppeling tussen een methode en URL en de code die moet worden uitgevoerd.                         |
| Router              | Onderdeel dat de juiste route zoekt en de code aanroept; bij ons <code>RequestHandler</code> .     |
| Same-origin policy  | Browserregel: JavaScript mag geen responses van een andere origin lezen.                           |
| Server              | Het programma of de computer die requests beantwoordt.                                             |
| SQL-injectie        | Aanval waarbij iemand eigen SQL-code in je query smokkelt.                                         |
| Stateless           | De server onthoudt niets tussen twee requests.                                                     |
| Statuscode          | Getal van drie cijfers dat zegt hoe een request ging, zoals <code>200</code> of <code>404</code> . |
| Token               | Geheime code waarmee een client laat zien wie hij is.                                              |
| URL                 | Uniform Resource Locator: het adres van een resource.                                              |
| URL rewriting       | Een URL achter de schermen omzetten naar een andere, bijvoorbeeld naar <code>index.php</code> .    |
| Validatie           | Controleren of de invoer van de client klopt.                                                      |
| Vanilla PHP         | Gewoon PHP, zonder framework.                                                                      |
| <code>vendor</code> | Map met de autoloader en pakketten van Composer.                                                   |
| XAMPP               | Pakket met Apache, MariaDB/MySQL, PHP en phpMyAdmin.                                               |